哈爾濱工業大學（深圳）
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

KFC-Agent 设计开发文档

低成本高准确率的多轮交互式 kdump 修复智能体

项目成员：

姓名	年级	联系方式（电话）
覃煜淮（队长）	大三	15577028319
岳亮	大三	13091688456
周健毅	大三	17876725670

项目导师：任玉鑫，张波

校内导师：夏文，李诗逸

2025 年 06 月

摘　要

随着 LLM 相关技术的发展，大语言模型在软件工程能力方面展现出优异的能力。然而，由于解决内核崩溃问题具有长上下文，上下文信息不足，领域知识繁杂，工具使用门槛高等特性，LLM 并未在解决内核崩溃问题上取得理想的效果。我们将介绍针对内核崩溃问题的 KFC-Agent（Kernel Fault Corrector-Agent）,不同于尝试单轮生成正确 patch 的 kgym 等相关工作，该智能体模拟人解决问题的实际场景，基于 docker 容器和 swe-agent 中的 swerex 模块搭建了一个交互的环境，实现了解决问题可利用的 bash,file_io,web_search,submit 四个工具，建立了一个 action-response 的多轮交互 agent 工作流，模拟开发者解决问题的流程，逐步收集信息并修改文件，最后提交完整 patch，有效地缓解了因代码导致的长上下文，上下文信息不足，工具使用门槛高等相关问题。在 kbench 这一 benchmark 上，KFC-agent 使用 deepseek-V3 模型，在生成两个 patch 的情况下解决问题的成功率达到了 11.83%,高于同等条件下 kgym 的 2.51%，在附加精确定位信息的情况下，解决问题的成功率达到 18.64%，以远远低于相关工作的实验成本和较少的 patch 数达到了较高的正确率。

本工作主要有以下贡献：

- 搭建了一个供 agent 多轮交互的环境，将代码仓库挂载在环境内由模型调用工具局部访问，有效缓解了因输入完整代码的长上下文问题
- 实现了非交互式命令行，联网搜索，文件操作，patch 提交四个工具，模拟了开发者真实解决问题使用到的工具，对针对内核问题的复杂工具进行了抽象，降低模型工具使用门槛的同时增强模型解决问题的能力
- 建立了一个多轮交互的 agent 工作流，让模型根据环境的反馈 response 以及上下文历史信息决定下一步采取的 action，模拟开发者实际的多轮交互解决问题情况，充分发挥模型探索收集信息的能力，有效缓解原有 crash report 上下文信息不足的情况
- 搭建了一个本地测试环境，为 patch 提供基于执行的正确性验证，解决了 kgym 测评环境只提供云端部署的问题，减少实验成本
- 探索了生成 patch 数和定位对执行情况的影响，验证了 patch 数增加对编译正确率和执行正确率都有帮助，定位对执行正确率提高有帮助

本项目赛题为 proj352-LLM-based-kdump-analysis，有以下三个目标：

- ✓ **目标 1（基础）：**借助 LLM 等手段自动化常规 kdump 分析流程；
- ✓ **目标 2（基础）：**构建理解 kernel 代码的 rag 框架；
- ✓ **目标 3（进阶）：**结合历史问题数据（来源自 bugzilla、maillist 等……）进行问题分析

初赛阶段，我们的赛题完成度如下：

目标	基本完成情况	额外说明
1	✓ 全部完成	1.基于 swe-agent 中的 swe-rex 搭建一个环境，供智能体访问代码与探索； 2.构建了四个工具供模型使用，模拟开发者解决问题流程； 3.设计一个与环境多轮交互的 agent，模仿开发者逐步收集信息解决问题； 4.参考 kgym 与 syzkaller，完成了测评环境的搭建，跑通测试流程
2	初步完成（≈40%）	1.对数据库所需数据进行了初步思考； 2.调研测试了当前效果较好的代码 embedding 模型；
3	初步完成（≈30%）	1.查看并确立了对历史数据的选取标准； 2.确立了历史数据对解决 kernel crash 问题的作用：用于理解问题思路，参与智能编排；
总计	初步完成（≈70%）	1. 完成理解 kernel 代码的 rag 框架并且使用历史数据获得思路，引入之后能编排 2. 算力允许的情况下探索 RL+agent，让 agent 学习解决问题的能力这一路径

目 录

1 概述 ····· 5
1.1 项目背景及意义 ····· 5
1.2 国内外研究概况 ····· 7
1.2.1 linux 内核崩溃处理 ····· 7
1.2.2 LLM 解决内核崩溃相关研究 ····· 9
1.3 项目的主要工作 ····· 13
2 需求分析 ····· 14
3 系统设计 ····· 14
3.1 系统整体架构设计 ····· 15
3.2 系统整体工作流 ····· 17
3.3 子模块设计 ····· 18
3.3.1 Environment ····· 18
3.3.2 Tools ····· 20
3.3.3 KFC-agent(多轮交互管理实现) ····· 21
3.3.4 测评环境 ····· 23
4 系统实现 ····· 24
4.1 KFC-AGENT 的执行流程 ····· 24
4.2 KFC-AGENT 模块实现 ····· 32
4.2.1 Environment 模块 ····· 32
4.2.2 KFC-Agent 工具调用模块——Scarffolds ····· 42
4.2.3 KFC-Agent 性能评估模块—— KERMIT ····· 54
5 系统测试 ····· 67
5.1 测试准备 ····· 67
5.1.1 数据集来源与背景 ····· 67
5.1.2 数据采集流程 ····· 67
5.1.3 数据集特性 ····· 69
5.2 测试结果 ····· 72
5.2.1 测评指标 ····· 73

5.2.2 总结果图 …… 74
5.2.3 patch 数对编译正确率与执行正确率的影响 …… 75
5.2.4 定位对执行正确率的影响 …… 77
6 初赛总结与决赛展望 …… 77
6.1 初赛工作总结 …… 77
6.2 创新点 …… 79
创新性多轮交互 Agent 工作流设计 …… 79
抽象开发者行为为可调用工具，降低 LLM 使用门槛 …… 80
构建支持真实源码交互的环境，缓解上下文长度瓶颈 …… 80
构建标准化测评闭环，实现修复效果自动验证 …… 80
探索 patch 生成数量与执行效果的关系 …… 80
结合定位信息辅助生成，提高问题修复的精准度 …… 80
6.3 决赛展望 …… 81
7 参考文献 …… 82

1 概述

1.1 项目背景及意义

KFC-agent 是一个基于 LLM 的自动化内核崩溃修复智能体，通过 RAG 检索技术完成具体文件修改定位。KFC-agent 全称 Kernel Fault Corrector Agent，处理内核崩溃问题如同 KFC 般便捷高效，项目仓库位于 https://gitlab.eduxiji.net/T202518123995935/project2721707-303140。项目背景与意义具体如下。

Linux 内核在运行过程中可能发生几种典型的崩溃现象，反映系统在面对错误时的不同处理方式：最常见的是 Kernel Oops，当内核检测到空指针访问、数组越界等非致命错误时，它会打印 oops 日志并终止相关进程，但系统整体仍可运行。如果错误更严重或系统无法继续运行，就会触发 Kernel Panic，此时内核会输出错误信息后立即停止或引起重启，用以防止系统进入不稳定状态 。

为了调试和分析，可以启用 kdump 机制，这会在 panic 发生时保留现场并载入一个次级 kernel，将内核内存转储为 vmcore 以便后续崩溃分析。该机制避免了 BIOS 重启流程，同时保证崩溃前内存内容不被覆盖，随后可以将核心转储写入本地 /proc/vmcore、内存文件系统或网络路径，供使用 GDB、Crash 等调试工具离线分析，为 Linux 系统深度诊断、崩溃原因分析提供了强有力的支撑。

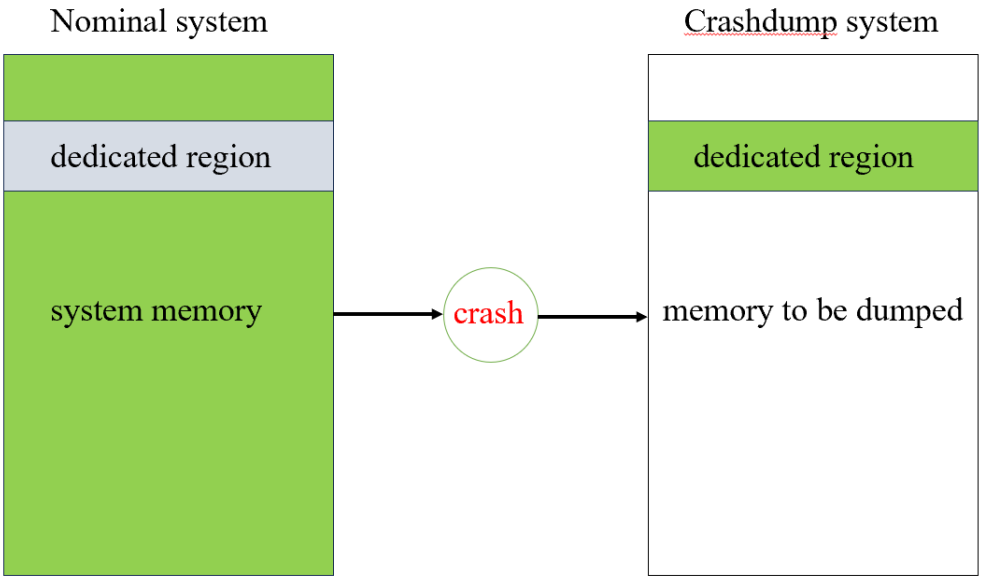


图 1 kdump 原理图

基于 kdump 的强大崩溃现场重现能力，kGym 方法将 crash reproducer、kbench 数据集与多核编译环境整合为一个端到端的评测平台，可以自动完成内核编译和 crash 重现，还能高效并行运行多个虚拟机实例，检测 crash 日志、检索相关源码并拼接 prompt，驱动 LLM 生成补丁建议；最后 kGym 自动重新编译并执行 reproducer 来验证 patch 效果。kGym 的设计仿照内核开发者的调试流程，同时提供了可扩展的 kBench benchmark 与并行执行能力，能以现实规模量化评估 LLM 在系统级崩溃修复上的表现，已成为解决内核崩溃的标准基线。

尽管 kGym 为系统级内核补丁生成构建了强大的评估平台，但其在实际 crash 修复任务中表现仍不尽如人意。在全自动模式下，应用单 patch 的 GPT4 模型执行成功率为 1.08%，而 gemini1.5 Pro 模型仅为 0.72%；即使在提供了相关源码片段的辅助模式下，成功率也难以超过 10%。这一结果暴露了多个瓶颈：首先，Linux 内核的规模庞大，且并发、安全与硬件耦合复杂，使得 LLM 在理解宏观语境、识别崩溃根本成因方面能力薄弱；其次，prompt 长度限制导致其只能观测一小部分上下文，缺少完整 trace 和代码信息；再者，当前模型仍难处理多线程与内核同步问题，生成补丁后常因语义不一致或逻辑不安全而引发新的错误。因此，kGym 框架亟需改进。

表 1.1 kGym 部分测试结果

应用 patch 数	GPT-3.5-Turbo	GPT4-Turbo	Claude3 Sonnet	Gemini1.5 Pro
1	0%	1.08%	1.79%	0.72%
10	0%	5.38%	NULL	3.58%

至此，我们简单介绍了 kdump 机制和 kGym 的实现。下面我们将系统阐述 KFC-agent 框架的研究意义。

第一，KFC-agent 缓解输入完整代码带来的长上下文及调用关系分析问题。我们构建了 agent 与内核代码交互环境 EnvSim，在容器中挂载内核代码并集成多种 tools 供智能体调用，从而赋予 LLM 自动阅读、修改整个内核代码的权限。

第二，KFC-agent 支持 LLM 多轮交互，且具备精准检索定位能力。通过设计 RAG 框架，在模型与环境交互前先根据 crash report 内容对要修改的文件进行初步定位，KFC-agent 在节省分析时间和调用次数的同时，提高了修改效率和准确性。

在项目的逐步完善中，我们获得了宝贵的实践经验。3-5月，我们针对syzbot网站的经典crash进行了广泛调研，并实现了crash_report的reproducer自动化复现；5月中旬大致完成了agent与交互环境的搭建，开始推进实验评估，同时完善修改位置定位和编译优化等功能，并推进文档的设计。

1.2 国内外研究概况

本小节主要介绍Linux内核崩溃处理系统架构、目前利用LLM解决内核崩溃的几种方法，以及使用强化学习提升LLM解决内核问题能力的方法，为搭建KFC-agent奠定基础。

1.2.1 linux 内核崩溃处理

Linux 内核崩溃处理系统是操作系统可靠性的核心保障机制，实现了从硬件异常检测到软件错误分析的完整故障管理链条。由于现代计算环境的复杂性，该系统发展出丰富的子模块协同体系，包括基于内存转储的 kdump 机制、面向实时诊断的 netconsole/pstore 日志系统、深度集成的 KGDB 调试框架，以及云原生的崩溃报告服务等。所有组件都遵循统一的事件响应模型，如图 2 所示的崩溃处理网络，其中硬件异常信号通过 NMI 中断触发双内核切换，内存镜像经 Crash 工具链解析后，最终与结构化日志系统协同生成可追溯的崩溃分析报告。

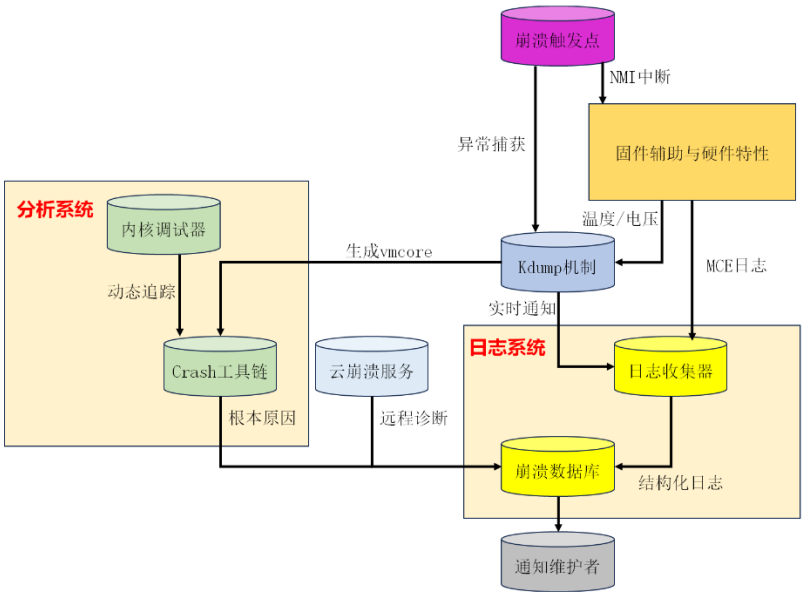


图 2 崩溃处理架构图

图中各个模块的具体功能如下：

- **内核崩溃触发源**

  当内核执行 panic()、BUG_ON() 断言失败或发生 oops 时，系统利用 NMI 中断立即打断 CPU 执行，触发跨 CPU 的 backtrace 或 panic 行为，收集崩溃信息。

- **硬件异常检测模块**

  模块集成了多种硬件层级的故障检测机制以确保系统健壮性，各硬件成员协作触发系统级错误处理。EDAC 实时监控内存控制器的 ECC 状态，当单比特错误被纠正或发现不可恢复错误时会触发警报或日志记录；MCE 捕获如内存总线故障、CPU 缓存错误或系统过热等硬件异常，生成硬件错误事件，由内核向日志或控制台报告；通过 mcelog 或 rasdaemon 等工具，这些事件可被解析并上传至 syslog，便于系统管理员监控。NMI Watchdog 利用不可屏蔽中断定期检查内核是否卡住，并在检测到硬锁死时打印调用栈或触发重启。

- **Kdump 崩溃转储机制模块**

  在发生 kernel panic 或 oops 时，系统通过预留内存启动一个独立的 capture kernel 接管当前运行环境，并将崩溃时的整个内存内容 vmcore 导出为 ELF 格式，以便后续分析调试。

- **日志系统模块**

  通过日志收集器收集内核崩溃与系统日志信息，在崩溃发生时异步补充上下文信息，或为远程诊断提供数据支撑。

- **崩溃数据库**

  将上下文日志、诊断信息等数据存入数据库，用于定位错误根本原因、统计崩溃频次等。

- **Crash 分析系统**

  利用转储文件 vmcore 来调试和诊断崩溃根本原因，通过符号解析、堆栈回溯还原崩溃过程。

- **云端崩溃服务模块**

  云服务自动捕获内核转储，将 vmcore 上传云端分析。

根据图 2 中的崩溃处理架构图，KFC-agent 从 crash report 入手进行分析，从而获取发生崩溃时系统详细的堆栈信息。

1.2.2 LLM 解决内核崩溃相关研究

近年来，随着大规模语言模型（LLM）在代码理解与生成能力上的突破，研究者开始尝试让这些模型参与到 Linux 内核崩溃的自动修复中。与传统的静态分析或手写规则方法不同，LLM 可以直接基于崩溃日志、内存转储、执行轨迹等输入，结合源码上下文进行语义理解，并生成修复补丁。这种方法赋予了系统崩溃修复机制类似人类专家的思维能力——不再仅依靠硬编码规则，而是利用自然语言对崩溃原因进行解释、提出假设、再继续生成修改代码。该方向虽仍处于初期阶段，但已经形成清晰定位、LLM补丁生成、验证闭环的完整流程体系，具有丰富的研究意义。

不同于常见的应用级bug修复，内核崩溃涉及上千万行 C/汇编代码、硬件交互、多线程并发等挑战。此类研究通常包括crash 重现环境搭建、崩溃日志 + 源码检索定位、将上下文严谨地拼接为LLM prompt、利用LLM修复生成补丁、以及补丁应用与实际验证。在这一方向上，谷歌的部分研究者已构建了面向 Linux 内核的代表性评估基准kGym和自动修复agent框架CrashFixer。下面将对这两项研究进行简单介绍。

- kGym[2]

kGym 是首个集成 LLM 的内核补丁自动化验证平台，也是一套完整的内核崩溃修复评估框架。平台将补丁验证拆分为编译、重现、日志分析、源码检索、patch 生成与验证等多个独立的阶段，并将每个 kernel compile + reproducer 运行作为并行执行单元，在多个 VM 实例中并发调度。每个完成的 crash 重现周期均作为一个质量评估维度，最终通过重现崩溃日志判断 patch 是否有效。该设计不仅兼容真实 Linux 源码和 kBench 数据集，也支持 unassisted与 assisted两种评估模式。

kGym采用“编译 → 重现 → 修复 → 验证”作为评估流程，在内核编译完成后利用预编译的磁盘映像和reproducer文件复现内核崩溃，崩溃时收集内核恐慌信息获取崩溃报告。同时由于许多bug不稳定，kGym的一个Reproduce操作会启动多个VM并行运行Reproduce-Bug，从而增加了再现内核崩溃的机会。这种设计不仅便于量化 LLM 在不同上下文下的修复效果，也便于统计失败率、评估性能瓶颈，并为后续增强策略

提供实验基础。

kGym的修复机制通过直接构造包含丰富相关上下文的Prompt，由先进的LLM生成修复程序。对于每个内核崩溃，在获取crash report后，找出所有相关文件，并从linux内核中找到对应上下文构造模型提示词，得到suggest的patch。

补丁提交之后，kGym会通过一套严格的“应用－编译－重现”校验机制来确认patch是否真正解决问题：首先，应用LLM输出的git diff格式补丁，并在指定commit-id的代码库中应用该变更，同时使用与initial build相同的参数重新编译Linux内核；接着，kGym调用Reproduce-Bug API，在编译完成的磁盘镜像上启动多个VM实例并行运行reproducer，这个过程中若任一实例在10分钟内未触发崩溃，即视为该补丁成功修复了crash。这种设计确保了补丁有效性的实时反馈闭环，并通过编译一致性和VM再现性保障了验证的客观性和可复现性。

kGym利用LLM解决内核崩溃的流程如图 3所示。图 3符合上文“应用－编译－重现”再验证的校验操作顺序。可见，为了再现错误，kGym首先编译有错误的内核版本并运行再现器文件。接下来，LLM将根据分析后的崩溃报告以生成潜在的补丁。每个代码补丁都被提供给kGym，然后kGym将补丁应用于有缺陷的内核版本，编译整个内核，再执行一个reproducer文件来检查bug是否已经成功解决。

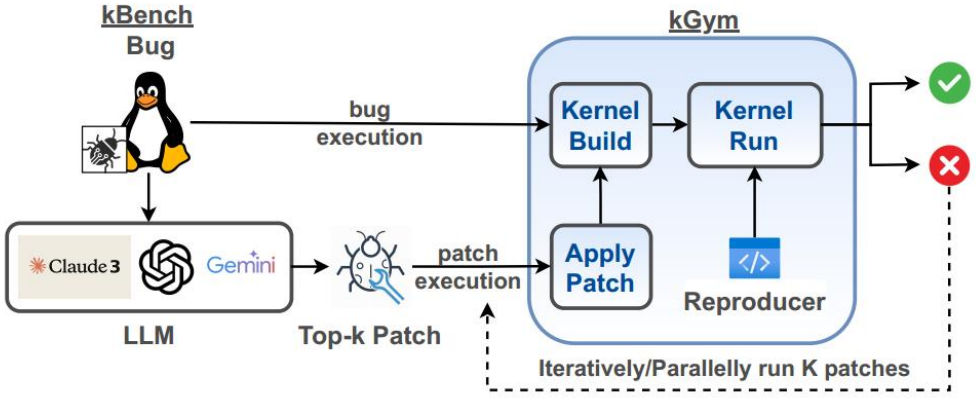


图 3 kGym 流水线

kGym在自动化评估平台设计方面树立了标杆，通过结构化流程和清晰基线为LLM 驱动的内核问题修复奠定了基础。但即便在即便在assisted模式下，LLM 补丁效果依然不佳，说明该框架在定位与修复复杂内核 bug 时理解能力仍不成熟，需要在增强上下文检索和语境理解、提升执行效率等方面持续优化。

- CrashFixer[1]

CrashFixer是谷歌团队在kGym基础上做出的高效改进。CrashFixer将LLM单步推理生成修复程序改为多轮agent思考，从而大大提高了系统解决内核问题的成功率。CrashFixer 提出“假设生成 + 补丁生成”的两阶段调试流程，模仿内核开发者的思路进行问题定位与修复。在假设阶段，系统通过最小化执行跟踪记录关键崩溃上下文，让 LLM 提出明确的崩溃原因假设；补丁生成阶段将该假设转化为具体的 replace-based 代码编辑，得到一系列候选补丁，并通过编译检查和多轮 self-consistency 验证，筛选出最符合假设逻辑的补丁。

CrashFixer为满足大规模Linux源码修复场景，构建了性能更优的kGymSuite平台，并引入selective execution trace 收集、分布式编译缓存SuiteCache、patch局部快速编译检测以及并行debug-tree/forest结构探索补丁路径等多个系统级优化策略。这些改进显著降低了反馈循环时延，并提升了并行补丁验证能力。

CrashFixer分析崩溃的过程中，首先会根据当前上下文生成一个崩溃的原因假设，再利用LLM推理能力选出具备高可能性的假设，用于作为Prompt生成修复程序，并与kGymSuite测试平台结合使用完成整个流程，如图 4 CrashFixer debug流程所示。

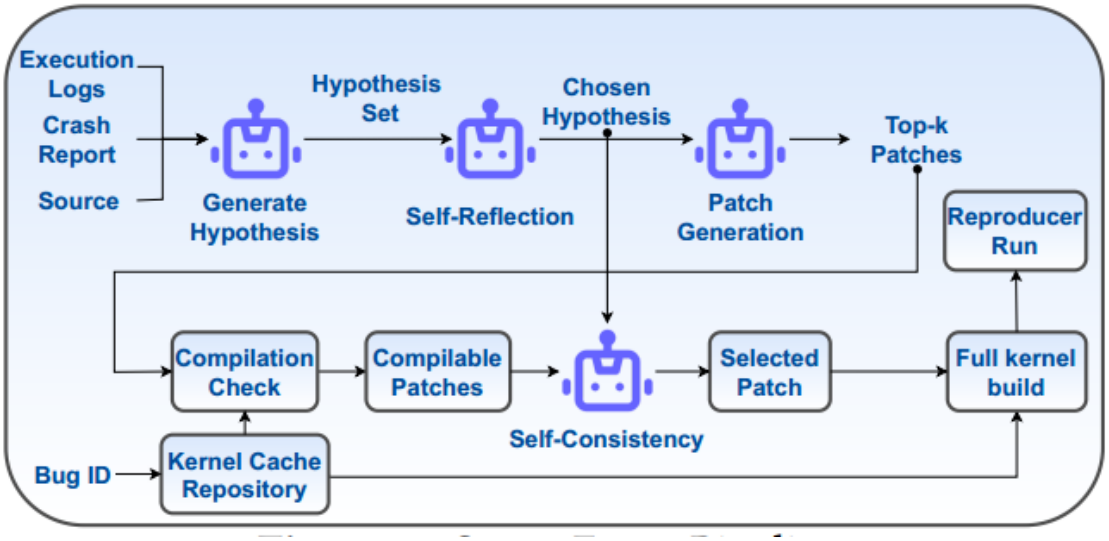
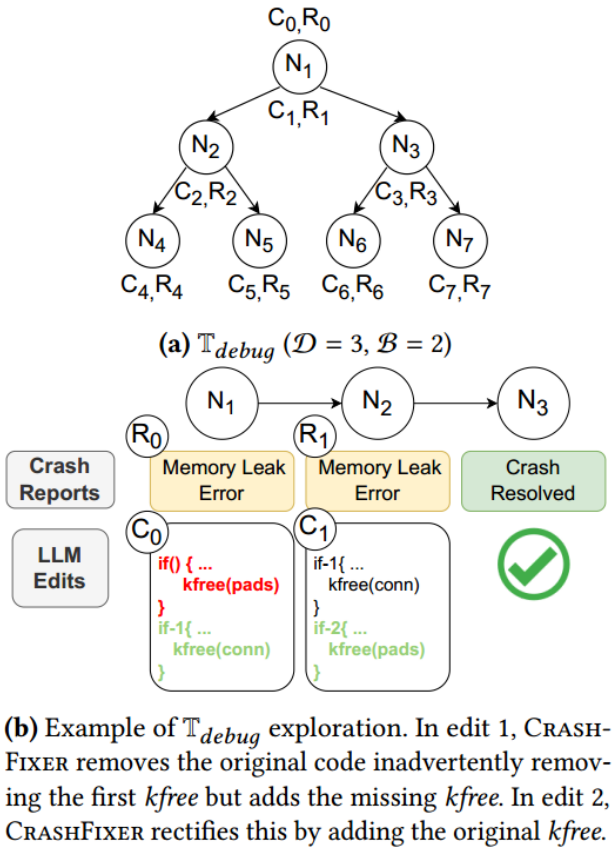


图 4 CrashFixer debug 流程

CrashFixer 将调试周期组织成包含多个并行实例的森林数据结构，以在“调试效率”与“探索广度”之间达到平衡。在这一设计中，每个实例或“树”从相同的初始 kernel 状态开始，独立执行一系列假设生成、补丁生成、编译与重现循环；这些并行路径可以抑制因单一路径中偶发 hallucination 导致的错误传递，从而形成多个修复

尝试。同时，若某条路径发现有效补丁，则该树停止扩展，其余路径继续探索，从而降低过早遗漏潜在修复的风险。这种森林结构不仅强化了 CrashFixer 处理复杂bug的能力，而且基于并发计算资源提升了对多样 candidate patch 的探索效率。



(b) Example of $\mathbb{T}_{debug}$ exploration. In edit 1, CrashFixer removes the original code inadvertently removing the first *kfree* but adds the missing *kfree*. In edit 2, CrashFixer rectifies this by adding the original *kfree*.

图 5 CrashFixer 调试周期表示

为了提升LLM生成补丁的可靠性、可解释性，CrashFixer提出了两种新的机制：

① **Replace-based Edit Interfaces**。CrashFixer 不要求LLM直接输出完整的 git diff，而鼓励模型生成一组 (original_snippet， replaced_snippet) 的编辑元组，并要求提供每个编辑的推理说明。这种结构化输出方式能够显著提升补丁解释性和审查效率。

② **Patch Selection**。在 LLM 生成多个候选patch后，CrashFixer 将通过 kGymSuite 快速编译检测剔除编译失败的补丁；接下来要求 LLM 根据之前生成的假设，对剩余补丁进行对齐选择，确保所选补丁与假设逻辑一致。这种轻量的review机制有效地提升了补丁的语义准确性和合理性，防止单一补丁因 hallucination 出错，同时降低回归修复误差。

此外，CrashFixer还利用kGymSuite平台实施了执行轨迹压缩，从 kdump 输出中

提取与崩溃线程相关、集中在关键函数附近的 trace，并通过 trace_printk 在进程上下文中进行进一步筛选，显著降低了冗余日志量，同时为 LLM 提供更精确的上下文。同时，还引入了高效的 SuiteCache 编译缓存机制，只编译修改的文件，将未改动模块的编译输出缓存复用，从而将补丁反馈循环的编译时间削减至原有的20%以下。

虽然CrashFixer解决了kGym的多个痛点，且在kBenchSyz数据集上取得了显著的提升，但也有一些严重缺陷：

① **功能截断问题。**作者在论文中提到，虽然 CrashFixer 修复崩溃成功率大大提升，但其修复策略往往采取“移除故障功能”方式，即通过删除或避开出问题的代码路径来消除崩溃。这种方式虽然规避了crash，但可能牺牲关键功能，导致系统行为与设计意图不一致。

② **严峻的资源占用问题**。CrashFixer 在每个补丁验证阶段会启动多个虚拟机实例，这些 VM 不仅需要独立运行内核和 reproducer，还要占用固定的内存和磁盘镜像，从而显著增加 CPU、内存和 I/O 负载。同时，CrashFixer 使用森林数据结构管理多个并行调试路径，不同路径的补丁生成和验证需同时占用系统资源，极大地放大了计算和存储资源消耗。

- Agent+RL[3][4][6][7][8][10]

经过广泛的调研，我们发现agent+强化学习（RL）这一通过环境交互来持续学习解决问题能力的研究路径，并认为这一方法能给LLM在内核修复问题上提供新范式。近期研究如MLE-Dojo框架通过虚拟化Gym环境，已证明RL代理能在Kaggle等封闭任务中通过迭代优化学习复杂问题解决策略。我们将这一理念引入 Linux 内核崩溃修复场景：构建一个强化学习环境，将补丁生成、编译、崩溃复现和日志反馈视作连续的决策步骤。其中，LLM agent 可尝试不同修复策略，根据是否触发 panic、是否成功复现等不同的执行结果中获得奖励与惩罚，并通过 trial-and-error 学习逐步提升调用工具、检测不稳定原因、调整补丁方式的能力。

然而，由于当前算力资源不足，且模型参数量小无法解决该问题，我们暂未利用强化学习方法在patch执行准确率上取得较好的提升，但认为这可能是有效的未来研究方向。

1.3 项目的主要工作

KFC-agent项目致力于实现比赛中的功能要求，我们的工作全部围绕赛题中所阐述的功能特性而展开，实现并完善了以下三个特性：

- **特性一：实现基于LLM的自动化kdump分析**

  构建挂载了linux内核的docker作为agent交互环境，并设计完善的命令工具调用机制，实现基本的全自动LLM修复内核崩溃流程。

- **特性二：构建理解kernel代码的rag框架**

  搭建数据库，在智能体开始编辑内核代码前先根据crash report信息完成初步修改定位。

- **特性三：结合历史问题数据进行崩溃分析**

  利用社区discussion搭建rag框架，为LLM debug提供更多相关信息。

2 需求分析

针对赛题要求分析，KFC-agent最基本的功能就是搭建起一个全自动的基于LLM的内核崩溃kdump分析、评估框架，从而将人类专家完全从debug过程中解放出来。因此，首要任务是搭建起“复现崩溃——分析推理——评估“流水线。

由于自动化流程需要赋予LLM阅读、修改整个内核代码的权限，我们需要实现agent在测评环境中的文件操作、提交patch等非交互式命令行工具。另外，由于要结合历史问题数据分析内核崩溃，我们需要参考agent联网功能的实现，从而在互联网上自由获取相关信息，提高LLM分析精准度。最后，如何搭建有效的RAG分析框架成了亟待研究的问题。

关于如何在KFC-agent中使用RAG方法，我们起初并没有清晰的思路。因此，我们咨询了华为的任玉鑫和张波老师。老师们提出，可以利用任意有效的方法，例如类似AST技术，明确内核中复杂的函数调用关系，或将代码归类分片，缩小修改位置的检索范围，提高LLM修改准确性。

总之，KFC-agent要实现全自动化的完整内核精准修复流程，而且要尽量占用较少的资源，降低处理用时的同时压缩api成本。在本项目实现中，我们不仅实现了必备功能，还成功实现了历史数据分析、定位等附加功能，同时占用的硬件资源和api调用成本均远低于谷歌的工作。

3 系统设计

3.1 系统整体架构设计

在完成了reproducer复现crash_report并且将测评环境搭建起来之后，我们调研了当前kernel_crash自动化解决现状，阅读了智能体解决软件问题的论文和相关项目，我们总结出kernel_crash自动化解决问题流程主要存在以下问题：

- ✧ **上下文问题**：从上下文长度角度，内核**代码量达数千万行**，远超当前模型处理上下文上限；从上下文难度角度，内核代码**缺少高层语义，代码之间联系复杂，上下文内容准确理解具有难度**；从上下文信息量角度，更多的隐式的信息不在已知上下文中给出，**需要去挖掘和扩充有用的上下文**
- ✧ **领域知识繁杂：内核相关知识门槛高，内容更复杂**，硬件知识以及日志等多种相关信息理解都需要一定的领域知识
- ✧ **工具**：增强智能体能力的有效办法之一是使用工具，然而解决内核问题**相关的工具使用门槛太高，并高度依赖经验**，LLM 很**难按照正确的格式输出调用指令，且输出指令错误或是难达到预期的效果**，从而影响工具解析调用
- ✧ **智能编排**：内核问题本身具有复杂性，解决过程步骤多，上下文长，对**有用信息无法准确回溯以及定位，甚至发生任务遗忘**，因此**让模型明确任务目的，形成任务解决方法步骤十分重要**

在对当前智能体相关综述以及重要工作进行了浏览之后，我们关注到不少 agent+RL 的文章对当前问题的解决或许有帮助：**通过强化学习，让智能体从跟环境交互中学习到使用工具，学习到解决问题的思路与策略，学习到领域相关信息**。由于算力的限制无法训练参数量足够的大模型，我们初赛阶段搭建了一个可以交互并且执行命令的环境，提供了四种解决问题使用的工具，并构建了一个可以调用工具与环境持续多轮交互来不断获取信息的智能体。

系统架构图如下：

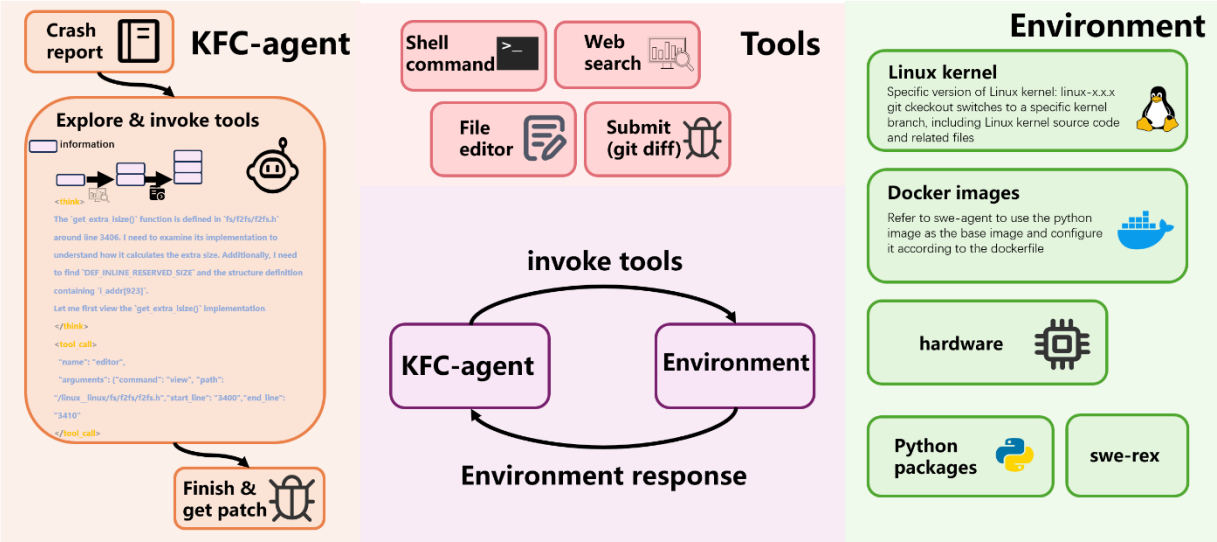


图 6 KFC-agent 整体架构图

该智能体具有以下优势：

1. **环境作为代码载体，解决原有工作因以代码为输入导致的长上下文问题**
2. **多轮交互探索挖掘信息，解决上下文原有信息不足以提供正确方案的问题**
3. **模拟开发者解决问题流程，对内核问题工具进行抽象，定义四种工具，解决了模型调用工具门槛**

- **Environment**

Environment本质上是一个挂载了linux仓库的docker容器，该容器使用已经提前配置好相关python包的swe:latest作为镜像，让智能体能够通过swerex[9]这一swe-agent[5]中的模块与环境通信，智能体与环境的通信通过reqeusts库中的post方法进行

环境解决了原有的长下文的问题，将代码仓库挂载到环境之中，模型只需要调用file_io工具查看文件即可获得特定行的上下文，极大减少了上下文长度；并且环境为智能体提供了一个持续交互获取信息的基础，有利于智能体把问题细分，不断获取信息进而解决问题

- **Tools**

Tools是agent与Environment交互的途径， agent调用LLM的API生成特定格式的

response，然后工具管理模块对response进行解析，从中提取出模型的思考过程和工具调用，根据解析出来的工具调用，通过function_calling执行特定的工具，特定的工具实现会基于swerex，进而让工具直接作用于环境，实现agent与环境的交互

- **KFC-Agent（Kernel Fault Corrector - Agent）**

  该智能体调用LLM的API，在每一轮交互过程中，针对特定的prompt以及交互上下文，给出当前思考过程和调用工具的response，经过工具解析和在环境执行中得到反馈，将执行+反馈加入上下文信息，再根据新的上下文做出新一轮的决策，直至模型觉得信息足以解决问题边修改文件提交patch。

3.2 系统整体工作流

该智能体整体执行流程如下，通过调用工具与环境多轮交互不断获取信息，最后对环境中的代码库进行修改并提交 patch，整体流程图如下：

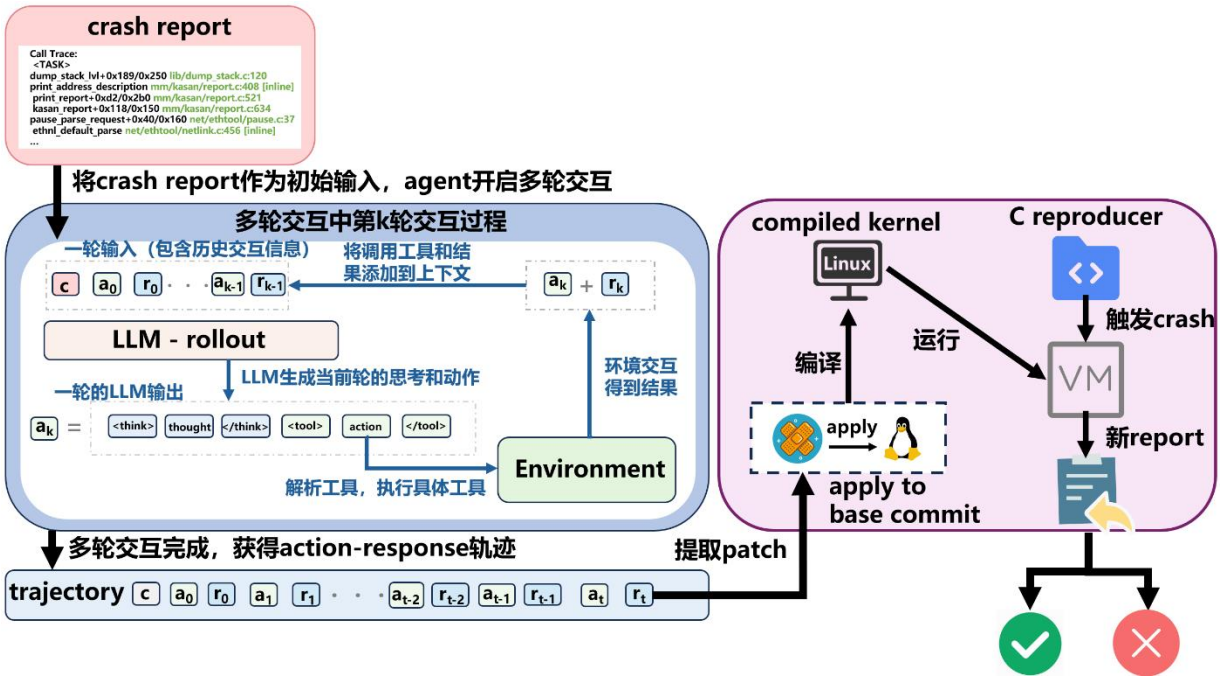


图 7 系统整体工作流示意图

- **rollouts生成trajectory的工作流**

  系统以crash_report作为输入，根据每一步之前的上下文，调用LLM的API，生成一个新的<think></think>和<tool_call></tool_call>，通过对生成的

<tool_call></tool_call>进行解析，获得调用工具的名称以及参数，然后工具会与环境进行交互，得到环境的反馈，并把LLM输出以及环境反馈加入上下文，开启下一轮的rollout，以此让模型能够持续从环境获取信息，逐步解决问题，最后生成一个trajectory，trajectory中的最后一个输出即submit的结果，也就是提交的patch

- **测评环境**

对patch正确性的测评是基于执行的测评，是最能体现patch正确性的测评。该测评环境以patch作为输入，将备份好的linux仓库切换到特定的版本，并且apply由大模型生成的patch，对得到的新内核代码进行编译并通过syzkaller[11][12]文档中给定的reproducer所使用的镜像进行运行，检测导致该crash_report的.c代码是否还造成crash_report，根据新的输出判断该Patch是否通过测试

3.3 子模块设计

本小节将介绍Environment，Tools的设计逻辑，KFC-agent的工作流设计逻辑以及测评环境的设计逻辑，具体的实现代码请参考第四部分

3.3.1 Environment

Environment的结构如图 8所示。

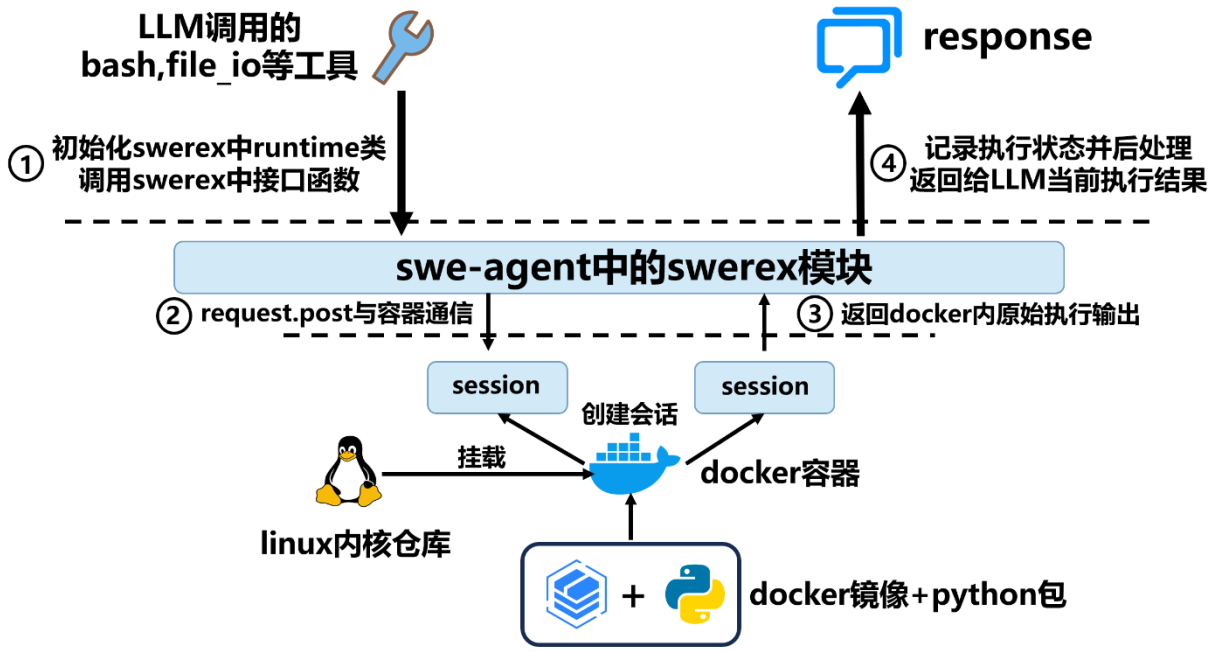


图 8 Environment 模块架构图

从environment结构图中我们可以知道，环境模块的层次结构可大致分为5个层次，自底向上可以分为：docker镜像+python包，挂载了linux内核仓库的docker容器，docker容器创建的session会话，swe-agent提供的swerex模块[9]，输入输出（工具和response等）

- **Docker镜像+python包**

  Docker镜像采用swe:latest，该镜像是基于swe-agent使用的python3.11镜像以及我们为了实现特定功能安装的python包而搭建起来的一个新镜像，当修改swerex模块代码时要重新更新一遍该镜像，我们实验中的容器全部基于该镜像建立

✧ **挂载linux仓库的docker容器**

Docker容器基于上文提到的swe:latest镜像构建，该容器作为我们交互的环境，将linux仓库挂载进入容器，让agent能够通过与docker容器的交互实现特定范围的linux代码的获取，解决长上下文的问题

✧ **session**

session即一个会话，是swe-agent中为了实现docker容器与外界通信所执行的操作，让docker容器创建一个会话，外界与docker环境的交互实际上是与一个有状态的会话的交互，模拟真实的我们使用命令行shell的方式

✧ **swerex**

swerex是swe-agent中的关键模块，其中包含了agent与环境交互通信的具体实现:swerex中的remote.py文件通过使用reqeust库中的request.post方法实现与docker容器的通信并获得返回值，实现了交互的核心逻辑：如何将动作传递给环境即docker容器，如何获取docker容器执行后得到的返回值

✧ **tools & response**

tools直接被function_calling调用，在tools的具体实现中会调用run_in_session函数，该函数基于swerex模块实现，负责让工具在docker容器的一个session内执行并且获得执行返回结果response

上述为Environment模块的架构介绍，通过层层调用完成环境的建立，给工具与智能体提供交互的接口，解决长上下文问题，并且让agent不断从环境中探索，为解决上下文信息不足的问题提供环境基础

3.3.2 Tools

Tools的整体结构如图 9所示。

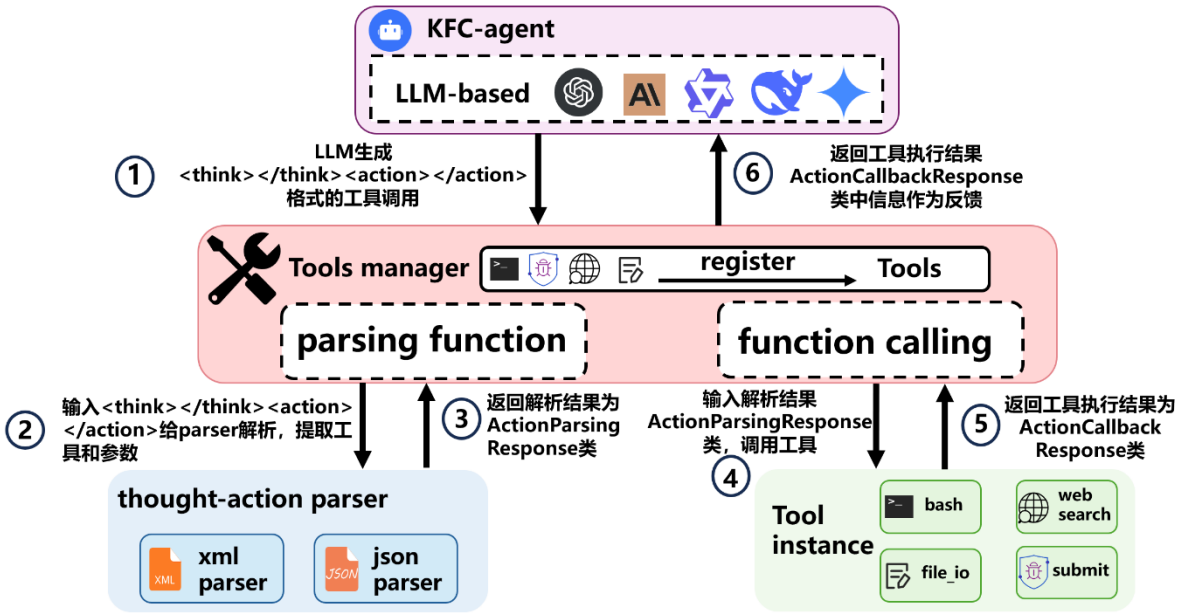


图 9 Tools 模块架构图

从图 9中可看到，tools manager类作为工具模块的核心，负责在整个工作流程中管理工具的使用，是tools模块中的核心模块;parser类负责从LLM输出的格式中提取出调用工具与参数，并返回解析的结果；ToolInstancel类即工具实例，检查解析命令并执行具体工具，返回执行结果；此外，图中还标注了数据执行过程以及数据形式的变化。下面将分四个部分介绍tools模块：tools manager;thought-action parser;tool instance;数据的在工具调用过程中的执行顺序与流动。

✧ **Tools manager**

Tools manager模块主要实现了三个函数：register函数负责注册工具，将工具实例（包含配置与参数等信息）加入环境之中供LLM-based agent根据特定格式调用；parsing function函数负责解析LLM-based agent的返回结果，根据generation_config中

指定的解析方式决定采用xml或是json格式解析，获得执行的命令与参数；function calling函数负责调用执行的工具，根据指定的工具和参数执行工具，获得执行的结果。

✧ **thought-action parser**

thought-action parser模块主要负责对LLM-based agent输入的xml或是json调用格式进行解析（KFC-agent实现方式采用的是xml格式解析），LLM-based agent会根据prompt要求输出<think></think><tool_call></tool_call>格式，而parser负责从LLM-based agent的输出中解析出<think></think>中的思考过程和<tool_call></tool_call>中的具体工具调用，并返回一个解析对象，供tool instance即具体的工具实例执行。

✧ **Tool instance**

Tool instance类为具体的工具实现类，实现了具体的工具执行代码，会接收已经解析好的工具调用对象，从工具调用对象中提取出具体工具的执行，然后根据命令和参数执行具体的工具，获得工具执行的结果。

✧ **完整流程的数据流动**

完整的一次工具调用流程中数据主要有三种形式的变化:工具调用由LLM-based agent在指定的格式下生成（KFC-agent指定格式为xml），数据形式为<think></think><action></action>；之后，tools manager调用parsing function函数，由parser将数据进行解析，返回ActionParsingResponse的数据类，包含thought，action，exit_status三种属性，其中action即解析出来的具体工具调用；最后，tools manager调用function calling 函数，Tool instance会根据ActionParsingResponse中的action实际工具调用，调用具体的Tool instance工具，执行获得ActionCallbackResponse结果，返回给LLM-based agent。

3.3.3 KFC-agent(多轮交互管理实现)

KFC-agent模块（多轮交互管理实现模块）结构如图3.3.3所示。

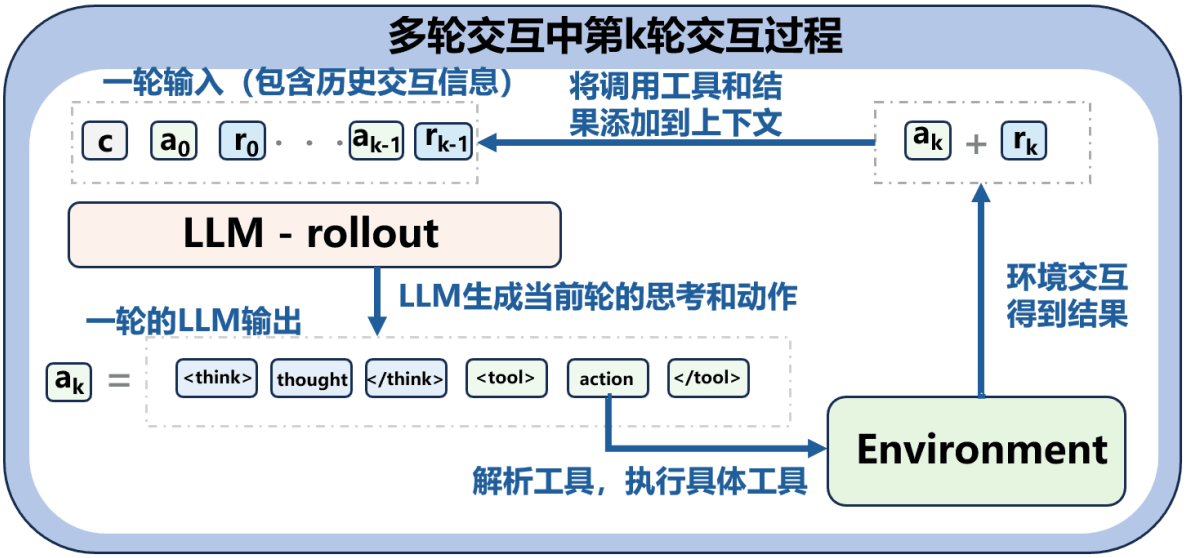


图 10 KFC-agent（多轮交互实现）示意图

由图 10可知，KFC-agent是一个LLM-based agent，其实现基于搭建好的环境和工具，KFC-agent生成patch的过程需要多轮rollout。每一轮rollout中，由LLM决定调用的工具，与环境交互执行，获得反馈后将执行和反馈作为上下文信息，开启新一轮的rollout。下面将以上图为例介绍一轮rollout的流程，主要包含四个部分：LLM generate，invoke tools，Environment response，message append。

- **LLM generate**

  LLM generate步骤以crash_report 和 之前所有的action+response的工具+反馈对作为输入，以产生的<think></think><action></action>作为输出，输出包含根据历史信息做出的思考和决定当前rollout执行的工具调用。

- **Delete Range**

  Invoke tools步骤以当前rollout产生的<think></think><action></action>作为输入，借助tools manager模块解析工具，从<action></action>中提取出调用的工具及其参数，整理成一个ActionParsingResponse对象，供之后调用具体的工具与环境交互。

- **response**

  response步骤以解析后获得的ActionParsingResponse对象作为输入，根据该对象中

的action属性决定调用的工具，与环境做交互后得到环境的反馈，将环境的反馈作为该步骤的输出

- **append**

  记录当前rollout的调用工具以及环境反馈，得到新的上下文，作为下一轮rollout的输入，指导LLM根据历史信息以及当前rollout的执行和反馈决定下一步的action

3.3.4 测评环境

✧ 测评环境整体架构图 11所示。

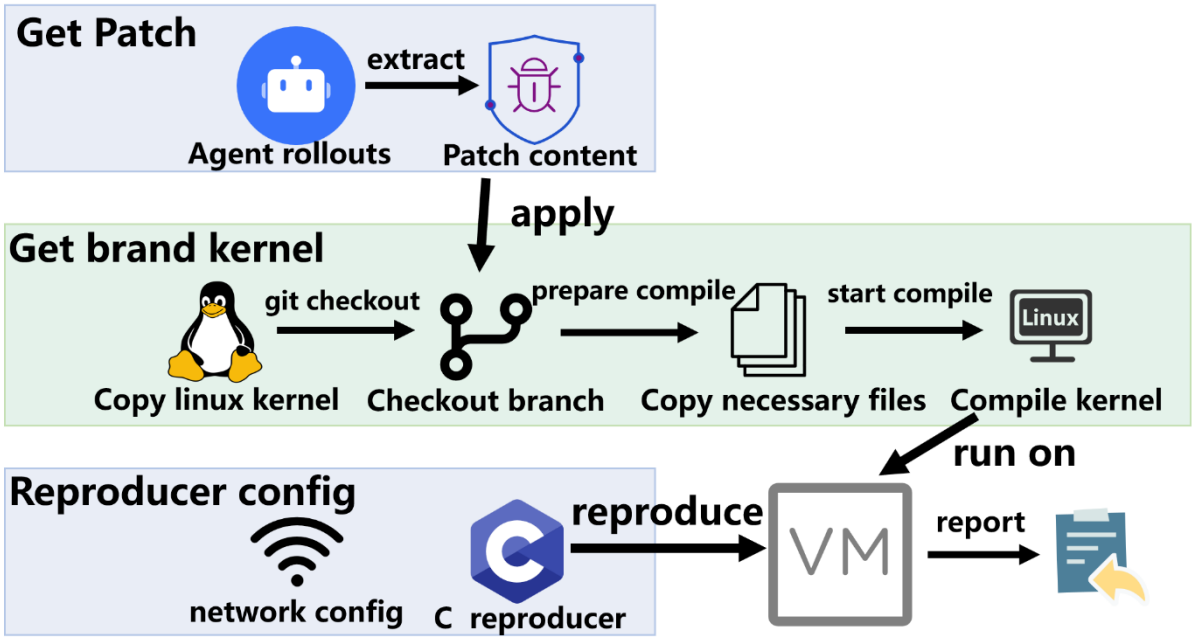


图 11 测评环境整体架构图

由图11可见，我们的测评环境模块主要三个部分组成：get patch即从智能体的rollouts中提取出patch的内容；get brand kernel即获得应用了patch之后的编译好的内核；reproducer config即对复现的配置。在完成以上三点之后，即可启动虚拟机并运行特定的内核，然后在虚拟机中用C reproducer复现错误，观察应用了patch之后的report，判断该patch是否有效解决原有kernel crash。以下分四个步骤简要介绍测评环境模块：get patch;get brand kernel;reproducer config;执行复现流程并判断patch是否正确

✧ **Get patch**

Get patch步骤主要是从LLM-based agent的rollouts中提取patch的内容，因为agent最后调用的工具都是submit，故可以从rollouts中的最后一轮rollout的response中提取出patch content。

✧ **Get brand kernel**

Get brand kernel步骤主要是完成特定分支应用了patch之后的内核编译。首先从特定的内核仓库路径复制一份linux仓库副本并在该出副本上进行后续操作，将linux副本切换到特定分支，应用get patch步骤获得的patch，得到新的内核源码。完成上述步骤后准备一系列之后运行虚拟机时必备的文件，完成之后开始进行内核的编译，得到特定版本linux源码应用了patch之后的新内核，用于测试patch的正确性。

✧ **Reproducer config**

Reproducer config 步骤主要是对一些复现必备的条件进行配置，比如获取C reproducer文件以及配置虚拟机网络，目的是使用导致错误的.c文件去在新的内核上复现，观察报告信息以便判断patch是否生效。

✧ **执行复现流程并判断patch是否正确**

该步骤主要是启动虚拟机，并且运行特定版本应用了patch之后的内核，并使用C reproducer去获得新的执行报告，并通过人为的严格的判断，将原有的crash report和得到的新report进行对比，最终判断patch是否生效。该验证方式是基于执行的patch验证，可信度高。

4 系统实现

在第3节中，我们对KFC-agent行了设计概述，本节我们将对KFC-agent的实现做出较为详细的描述。第4节将与第3节内容高度相关联，我们将在4.1中先描述代码整体流程，即生成patch的工作流，介绍流程中关键函数实现了什么功能，再在4.2模块介绍中详细介绍各个模块的关键函数。

4.1 KFC-agent 的执行流程

在正式开始各模块实现介绍之前，我们首先给出整体的代码工作流，提供一个整体的视角，便于理解之后各个模块实现的目的。需要说明的是，本节仅解释在工作流中关键的函数

- 脚本调用

在scripts/inference/run_server，sh脚本中指定了对应配置文件和数据文件，调用run_server.py函数执行逻辑

1.GENERATION_CONFIG指定了KFC-agent推理相关参数，如生成格式，最大对话轮数，生成patch数，输出路径等相关信息

2.ENVIRONMENT_CONFIG指定了交互环境和工具的相关参数，如环境基于的镜像，工具的配置文件，解析工具的格式

3.REPO_PATH和DATA_FILES分别表示需要挂载的仓库所在的路径和数据文件的路径

```
GENERATION_CONFIG="./configs/generation_config_server.yaml"
ENVIRONMENT_CONFIG="./configs/environment_config.yaml"
REPO_PATH="./Repositories/Raw"
DATA_FILES="/data/os/workspace/dataset/kbench.parquet"
MODEL_PATH="Qwen/Qwen2.5-14B-Instruct"

python run_server.py \
    --generation_config_path $GENERATION_CONFIG \
    --environment_config_path $ENVIRONMENT_CONFIG \
    --repo_path $REPO_PATH \
    --data_files $DATA_FILES \
    --model_path $MODEL_PATH
```

- run_server.py执行流程

在调研过程中发现许多让智能体从跟环境交互学习解决问题能力的文章，我们认为如果让模型在解决kernel_crash这一问题上训练提升模型解决问题的能力是有用的，但在做过些许尝试之后以失败告终，故不解释参考verl这一仓库进行强化学习训练的相关代码。

① 对参数解析，生成一个generation_manager对象，负责推理相关的内容执行

```
    generation_manager = GenerationManager(
            tokenizer=tokenizer,
            api_config=api_config,
            config=GenerationConfig(**generation_config),
            environment_config=env_config,
            repo_base_path=args.repo_path
        )
```

由于generation_manager对象在推理过程中比较重要，下面以表格形式介绍其输入参数：

变量名	所属类	变量描述
tokenizer	AutoTokenizer	用于文本分词的tokenizer实例，将文本输入转换为模型可以理解的token ID，以及将模型生成的token ID解码回文本
repo_base_path	str	本地代码仓库的基础路径。在每次生成（一轮rollout）任务中，相关的任务代码仓库会基于此路径进行克隆和挂载
config	GenerationConfig	核心配置对象，包含了整个生成过程的各种参数和设置，详细信息如上文脚本调用解释
environment_config	Dict	包含了用于配置交互式环境(Environment) 的信息，详细信息如上文脚本调用解释
api_config	Dict	包含了用于配置调用 LLM API 的信息，如base_url，api_key等信息

在对 generation_manager 类进行初始化的时候主要完成了以下两个功能：

（1）**处理并加载工具配置**

（2）**初始化环境配置 (self.env_config，此处对工具管理模块进行了初始化，详细介绍参考 4.2 环境部分介绍)**

② 数据集的构建与加载

```
    dataset = RLHFDataset(
        parquet_files=parquet_files,
        tokenizer=tokenizer,
        prompt_key="crash_report_link_content"      if      is_linux_kernel      else
"problem_statement",
        max_prompt_length=32768,
        filter_prompts=True,
        return_raw_chat=False,
        chat_template_func=None,
        truncation='error',
        filter_overlong_prompts=True
    )
```

```
    dataloader = StatefulDataLoader(
        dataset,
        batch_size=1,
        sampler=sampler,
        num_workers=1,
        pin_memory=True,
        persistent_workers=False,
        collate_fn=collate_fn
    )
```

该步骤主要实现了三个功能：

（1）从特定路径读取数据

（2）构造dataset对象（其中prompt_key即最终输入给LLM的初始上下文）

（3）构造dataloadaer对象

3 构造多个rollouts并行执行，生成patch并且评估

对kbench中的数据逐条处理，每条数据主要进行以下两步：

（1）运行 agent与环境交互循环（generation_manager.run_llm_loop）

```
    _,      gen_batch_output      =      generation_manager.run_llm_loop(gen_batch,
global_steps=step)
```

（2）评估生成patch的正确性（sub_reward_correctness_kbench），参考了verl原

有仓库设计的奖励代码架构以及swe-bench对patch正确性评估

```
    if is_linux_kernel:
        results = asyncio.run(sub_reward_correctness_kbench(
            rollouts,
            args.model_path,
            batch_indices=batch_indices,
            round_indices=round_indices,
            log_config=log_config
        ))
```

由于run_llm_loop和sub_reward_correctness_kbench是整个agent工作流最核心的两个函数，实现了多轮交互以及patch评估的功能，因此接下来将在执行模块简单介绍这两个函数的执行流程

- run_llm_loop执行流程

输入参数

变量名	所属类	变量描述
gen_batch	DataProto	包含了当前批次需要 LLM 处理的初始数据，即kbench中的一条数据
global_steps	int	表示当前为kbench中的第几条数据，便于日志记录

输出

表4.2.4 HOIT_RAW_HEDAER核心变量

变量名	所属类	变量描述
message_write_list	DataProto	包含了当前批次需要 LLM 处理的初始数据，即kbench中的一条数据
global_steps	int	表示当前为kbench中的第几条数据，便于日志记录

功能

完整的 LLM agent 交互循环，从接收初始问题到生成多轮交互并记录最终结果，支持调用LLM API以及本地训练，下图为调用API的单轮交互。

```
    if self.use_api:
        activate_message_list = self.preprocess_api(activate_rollout_list)
        print(f"Turn {step}, node {node_rank} calling API for {len(activate_message_list)} rollouts")
        responses = self._generate_with_api(activate_message_list)
        meta_info = gen_batch.meta_info
    assert len(responses) == len(activate_indices)
    updated_activate_indices = asyncio.run(self.stepwise_rollout_update(
        responses=responses, rollout_list=rollout_list, activate_indices=activate_indices
    ))
```

主要函数

下面根据函数执行模块简单介绍多轮交互的具体流程（包括上下文管理，调用API，与环境交互，更新状态的流程），其余不重要的如日志记录以及初始化，次要变量维护相关的函数此处省略

1 preprocess_api函数

输入参数

变量名	所属类	变量描述
activate_rollout_list	List[List[Dict]]	表示当前步骤仍在继续执行交互的rollout的历史信息列表

输出

变量名	所属类	变量描述
activate_messages_list	List[List[Dict]]	一个rollout的具体信息，内部为OpenAI所期望的对话格式

功能

将内部的 Rollout 对象中记录的 LLM-环境交互历史，适配并转换为外部大语言模型 API 所期望的对话消息格式，作为下一轮 step 交互的信息输入

2 _generate_with_api 函数

输入参数

变量名	所属类	变量描述
activate_message_list	List[List[Dict]]	外层列表的每个元素代表一个独立的 LLM 交互会话（一个 rollout）。内层列表

	是为该会话准备好的、符合外部 LLM API 格式的对话消息历史

输出

变量名	所属类	变量描述
response	List	列表内部的每个元素对应一个rollout基于历史信息对于当前step执行的回答，包含<think>和<tool>

功能

利用外部大语言模型 API 来生成对给定对话历史的响应，从中提取出思考的部分以及工具调用的部分

3 stepwise_rollout_update 函数

输入参数

变量名	所属类	变量描述
responses	List	包含了 LLM 对每个活跃会话最新一轮的生成响应，即思考和工具调用的文本
rollout_list	List[List[Dict]]	一个列表，包含了所有 Rollout 对象。这些 Rollout 对象存储着每个任务从开始到当前的所有交互轨迹和状态
activate_indices	List	一个整数列表，包含了当前仍然处于活跃的 Rollout 对象在 rollout_list 中的原始索引

输出

变量名	所属类	变量描述
updated_activate_indices	List	包含了在本轮交互结束后，仍然处于活跃的 Rollout 对象在 rollout_list 中的索引

功能

异步地驱动 LLM agent 与环境的每一步交互，并更新每个 Rollout 对象的内部状态和对话上下文。它处理 LLM 生成的响应，将其传递给相应的环境进行执行，然后根据环境的反馈更新 rollout 的历史记录，并确定哪些 rollout 还需要继续进行下一轮交互

- sub_reward_correctness_kbench函数

输入参数

变量名	所属类	变量描述
rollouts	List[List[Dict]]	包含了一系列 rollout 对象。每个 rollout 对象都代表一个 LLM 代理与内核环境交互的完整轨迹或会话，包含了生成代码、环境反馈等信息
model_path	str	使用的模型（LLM）的路径(仅针对本地推理)
batch_indices	List[int]	与 rollouts 列表一一对应。每个整数表示对应 rollout 所属的原始数据批次（batch）的索引，表示第几条数据
round_indices	List[int]	与 rollouts 列表一一对应。每个整数表示对应 rollout 在其所属批次中的轮次（round）索引，表示某条数据的第几轮生成
log_config	dict	包含了日志相关的配置信息。包括日志文件的路径、项目名称、实验名称，用于定位到具体的日志效果

输出

变量名	所属类	变量描述
rewards	List	每个 rollout 的验证结果，1表示测试通过，0表示测试失败

功能

执行针对 kbench 数据集任务的并行正确性验证。接收一系列的 rollout 对象（每个都包含一个 LLM 代理通过与环境多轮交互解决内核问题得到的 patch），然后为每个 rollout 启动一个独立的异步验证任务，验证当前 rollout 所生成 patch 是否解决当前问题

```
1.  async def sub_reward_correctness_kbench(rollouts, model_path: str,
batch_indices: List[int], round_indices: List[int], log_config: dict):
2.      """
3.      并行处理多个 rollouts 的验证
4.
5.      Args:
6.          rollouts: rollout 列表
7.          model_path: 模型路径
```

```
    8.          batch_indices: 数据批次索引列表
    9.          round_indices: 轮次索引列表
    10.         log_config: 日志配置信息
    11.
    12.     Returns:
    13.         list: 每个 rollout 的验证结果
    14.     """
    15.     start = time.time()
    16.     tasks = []
    17.     for rollout, batch_idx, round_idx in zip(rollouts, batch_indices,
round_indices):
    18.         task = asyncio.create_task(verification_kbench(
    19.             rollout=rollout,
    20.             model_path=model_path,
    21.             batch_idx=batch_idx,
    22.             round_idx=round_idx,
    23.             log_config=log_config
    24.         ))
    25.         tasks.append(task)
    26.     rewards = await asyncio.gather(*tasks)
    27.     print("[Kbench Outcome Reward] All instances run time: {:.2f}
seconds".format(time.time() - start))
    28.     return rewards
```

该函数对多个 rollout 进行测评并给出结果，其中主要对单个 patch 执行测评的函数为 verification_kbench，该函数和 eval.sh 脚本的详细介绍均在 4.2.3 测评模块中讲解

4.2 KFC-Agent 模块实现

本小节将对KFC-Agent模块的实现进行较为详尽的介绍，介绍内容将与3.2小节高度对应，需要说明的是，对于较为简单的实现，我们仅一笔带过；而对于实现关键点，我们将以流程图的方式呈现。

4.2.1 Environment 模块

本小节将讨论基于 swe-agent 建立的 Environment 类，主要分为四个部分介绍环境模块：

（1）环境的配置

（2）环境的管理

（3）环境中代码仓库的挂载

（4）环境多轮交互逻辑

- **环境的配置**

环境配置主要介绍环境如何设置基本配置信息

1 Environment 类

参数列表

变量名	数据类型	变量描述
deployment	**AbstractDeployment**	docker部署相关的对象，包含镜像信息等等，本仓库代码采用swe:latest作为镜像
tool_manager	**ToolManager**	工具管理类，实现工具与环境交互的逻辑
tools	**List[str]**	各个工具的名称

代码

```
class Environment:
    def __init__(
        self,
        deployment: AbstractDeployment,
        tool_manager: ToolManager,
        tools: list[str],
        meta: Any=None
    ):
        super().__init__()
        self.deployment = deployment
        self.tool_manager = tool_manager
        if isinstance(tools,  list):
            for tool_config in tools:
                self.tool_manager.register(tool_config)

        self.registered_repo = None
        self.meta = meta
```

2 EnvironmentConfig 类

参数列表

变量名	数据类型	变量描述
deployment	**DeploymentConfig**	配置类的对象，从配置文件中读取信息，包含部署相关配置信息
tool_manager	**ToolManagerConfig**	工具管理类配置文件，工具类的相关配置信息
tools	**List[str]**	各个工具的名称

代码

```
class EnvironmentConfig(BaseModel):
    deployment: DeploymentConfig = Field(
        default_factory=lambda: DockerDeploymentConfig(image="swe:latest")
    )
    tool_manager: ToolManagerConfig = Field(default_factory=lambda: ToolManagerConfig(strict=True, parsing_function="xml"))
    tools: list[str] | None = None
```

3 from_config 函数

输入参数

变量名	数据类型	变量描述
config	**EnvironmentConfig**	环境的配置信息

输出

变量名	数据类型	变量描述
cls	**Environment**	一个带着配置信息的Environment对象，之后环境具体的执行基于当前对象储存的基本配置信息

代码

```
def from_config(cls, config: EnvironmentConfig) -> Self:
    config = config.model_copy(deep=True)
    return cls(
        deployment=get_deployment(config.deployment),
        tool_manager=ToolManager.from_config(config.tool_manager),
        tools=config.tools,
    )
```

4 关系流程图

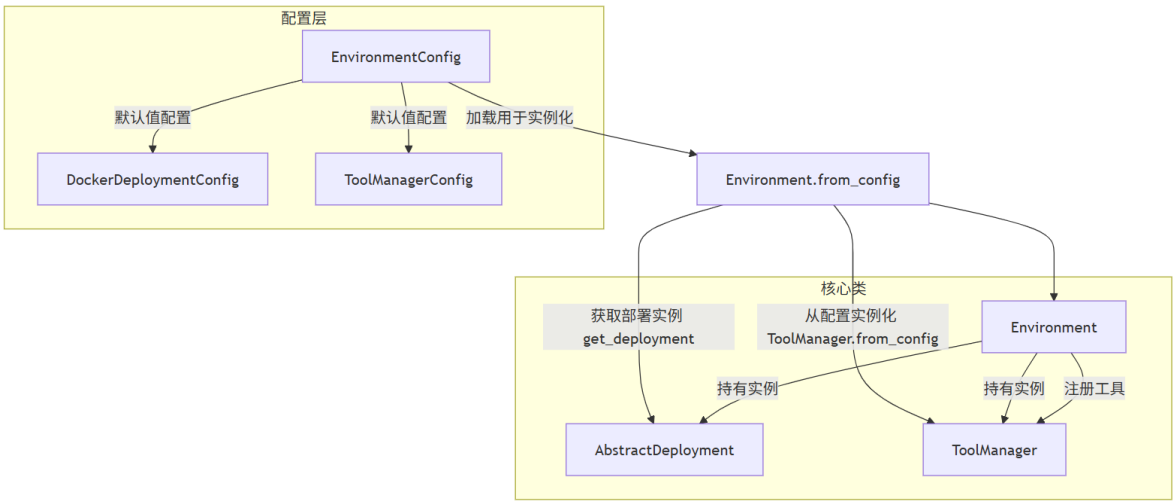


图 12 关系流程图

- **环境的管理**

环境管理部分主要介绍环境的启动，会话的初始化以及最后会话的结束和环境的销毁等内容，该部分整体代码如下：

```
    def start(self) -> None:
        import time
        start = time.time()
        print("start func: {}".format(start))
        asyncio.run(self.deployment.start())
        print("deployment: {}".format(time.time()-start))
        self._init_environment(session="default")
        self._init_environment(session="submit")
        print("submit: {}".format(time.time()-start))

    def close(self) -> None:
        asyncio.run(self.deployment.runtime.close_session(CloseSessionRequest(session="default")
))
        asyncio.run(self.deployment.runtime.close_session(CloseSessionRequest(session="submit")))
        asyncio.run(self.deployment.stop())
```

由于环境的搭建基于 swe-agent 中的 swerex 模块，因此在此展示一下 swerex 中

create_session 如何在环境中启动会话

（1）create_session 函数指定输入类输出类（swerex 实现），调用_request 方法

```
async def create_session(self , request: CreateSessionRequest) -> CreateSessionResponse:
    """Creates a new session."""
    return self._request("create_session", request, CreateSessionResponse)
```

（2）_request 调用 request 库中 reqeust.post 方法实现远程通信

```
def _request(self, endpoint: str, request: BaseModel | None, output_class: Any):
    """Small helper to make requests to the server and handle errors and output."""
    if hasattr(request, 'timeout') and request.timeout is not None:
        timeout = max(request.timeout + 1, 10)
    elif endpoint == "close_session":
        timeout = 60
    else:
        timeout = 120
    response = requests.post(
        f"{self._api_url}/{endpoint}", json=request.model_dump() if request else None, headers=self._headers, timeout=timeout
    )
    self._handle_response_errors(response)
    return output_class(**response.json())
```

下面介绍该部分的关键函数：

1 self.deployment.start()

输入参数

无

输出

无

功能描述

该函数执行了：准备 Docker 镜像，配置容器参数，启动容器，连接远程服务四步，把一个配置好的远程代码执行环境（swe:latest 镜像），封装在一个 Docker 容器里，将容器启动起来，让外部可以连接和交互

2 self._init_environment()

输入参数

变量名	数据类型	变量描述
session	**str**	表明一个独立的bash会话的标识符

输出

无

功能描述

该函数执行了：创建一个远程 Bash 会话并加载启动脚本，设置会话的字符编码环境变量两步，为 agent 创建一个新的隔离的远程 Bash 终端会话，并确保该会话加载了标准的 Bash 配置 (.bashrc)，并且设置了正确的 UTF-8 字符编码，为后续所有在该会话中执行的命令提供一个稳定可靠的运行环境

3 self.deployment.runtime.close_session()

输入参数

变量名	数据类型	变量描述
CloseSessionRequest(session="default")	**CloseSessionRequest**	一个 swerex 中确立的类的实例对象，一个对远程环境的请求消息

输出

无

功能描述

调用 swerex 模块中的 close_session 函数，通过 request 方法与远程环境通信，关闭当前会话

4 self.deployment.stop()

输入参数

无

输出

无

功能描述

该函数执行了：关闭远程运行时连接，停止并清理 Docker 容器进程两步，干净且健壮地关闭远程环境，并且做必要的后处理

5 关系流程图

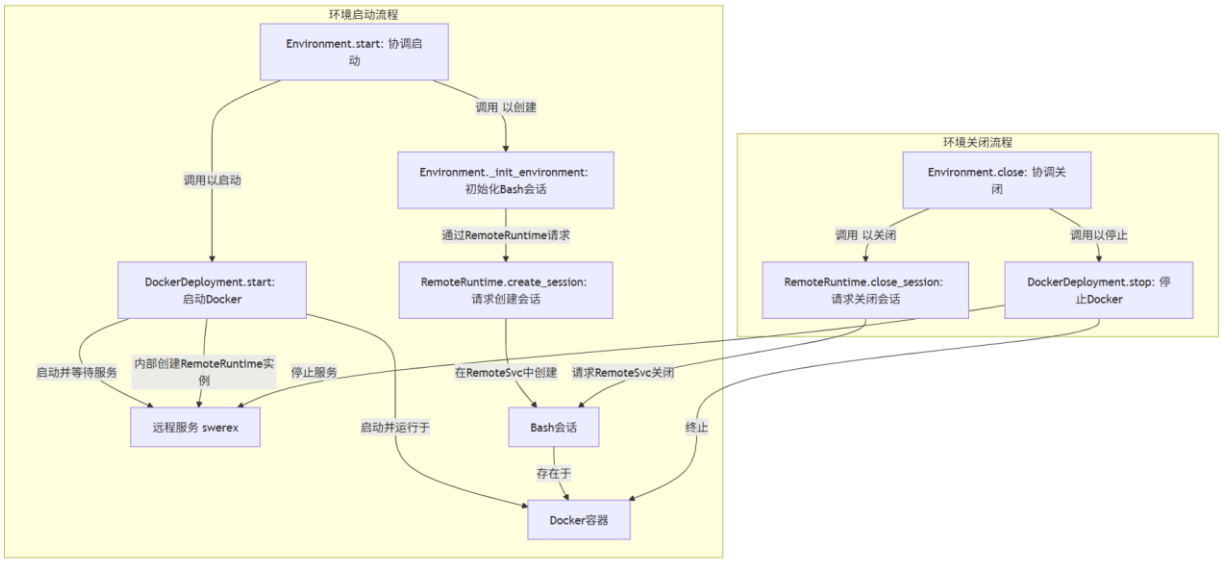


图 13 关系流程图

- **环境中代码仓库的挂载**

环境中代码仓库的挂载部分主要介绍了在远程环境成功部署在容器内部并且初始化会话之后如何将仓库进行挂载，该部分整体代码如下：

```
1.  def mount_repository(self,  repo: Repo):
2.      if repo is not None:
3.          repo.copy(self.deployment)
4.          self.registered_repo = repo
5.      else:
6.          raise ValueError("Repository cannot be None.")
7.
8.  def reset_repository(self):
9.      if self.registered_repo is not None:
10.         startup_commands = [
```

```
            f"cd /{self.registered_repo.repo_name}",
            "export ROOT=$(pwd -P)",
            "git status",
            "git restore .",
            f"git reset --hard {self.registered_repo.base_commit}",
            "git clean -fdq",
        ]
        asyncio.run(self.deployment.runtime.run_in_session(BashAction(
            command=" && ".join(startup_commands),
            check="raise",
            error_msg="Failed to clean repository.",
            timeout=120
        )))
    else:
        raise ValueError("No repository mounted. Please mount a repository first.")
```

下面介绍该部分的关键函数：

1 copy 函数

因为 linux 仓库代码量巨大以及历史提交多的特殊性，原有 swe-agent 提供的挂载方法不再适用，因此挂载部分主要对该函数进行修改，以满足我们代码执行并行，适配 linux 仓库两大需求

输入参数

变量名	数据类型	变量描述
self.deployment	**AbstractDeployment**	部署类实例化的对象，从中可以获得挂载的仓库名称等必要信息

输出

无

功能描述

本地代码仓库复制到一个临时位置，进行一系列 Git 相关处理，然后将其上传到远程执行环境，并在远程环境中进行初步的 Git 配置

代码简介

在 swerex 中 linux 仓库太大，处理具有特殊性，下面将讲解 linux 代码处理流程，因篇幅限制不具体介绍

（1）设置 uuid，为并行提供条件

（2）复制 linux 仓库作为副本，接下来在副本中进行操作

（3）切换到导致 crash_report 的特定提交分支

（4）删除.git 文件，把所有历史提交相关信息全部删除，避免上传时超出内存

（5）重新初始化一个 git 仓库，确保与 swerex 中其他仓库上传逻辑一致

（6）将重新初始化好的仓库上传

（7）对远程环境中的仓库配置 git 信息，便于能在远程环境进行 git 相关 git 操作来获得 patch，完成仓库的挂载配置

- **环境多轮交互逻辑**

环境多轮交互逻辑主要负责利用其中的工具管理模块对模型的响应进行后处理，并且调用根据解析出来的指令调用特定的工具，该部分整体代码如下：

```
1.  async def interaction(self,  input:  str)  ->  tuple[ActionParsingResponse,
Union[ActionCallbackResponse,  None]]:
2.      processed_response = self.tool_manager.response_postprocess(input)
3.      if processed_response.exit_status == False:
4.          return processed_response,  None
5.      response = await self.tool_manager.function_calling(
6.          action=processed_response.action,
7.          **self.env_variables()
8.      )
9.      return processed_response,  response
```

1　interaction 函数

输入参数

变量名	数据类型	变量描述
input	**str**	模型反馈的下一步调用的工具

输出

变量名	数据类型	变量描述
processed_response	**ActionParsingResponse**	解析出来的标准工具调用格式，包含调用的工具和相关的参数
response	**ActionCallbackResponse**	解析出来的工具在环境中执行完

	成后得到的环境的反馈

功能描述

处理传入的原始输入，将其解析为可执行的动作，然后调用相应的工具函数来执行这个动作，并返回解析结果和执行回调结果

代码

```
async def interaction(self, input: str) -> tuple[ActionParsingResponse, Union[ActionCallbackResponse, None]]:
    processed_response = self.tool_manager.response_postprocess(input)
    if processed_response.exit_status == False:
        return processed_response, None
    response = await self.tool_manager.function_calling(
        action=processed_response.action,
        **self.env_variables()
    )
    return processed_response, response
```

由于调用的两个函数 response_postprocess 与 function_calling 为工具调用模块内容，此处不加以解释

2 关系流程图

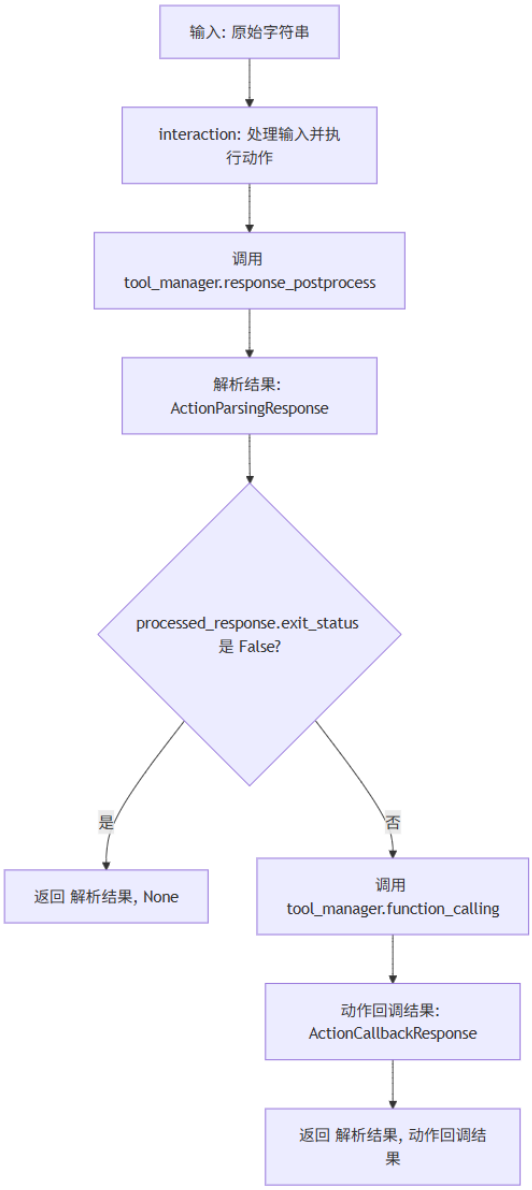


图 14 关系流程图

4.2.2 KFC-Agent 工具调用模块——Scarffolds

1. 模块整体架构

在本节中，我们将详细介绍 Scarffolds 工具管理框架的技术实现。Scarffolds 采用了分层架构设计，可以理解为一个"工具调用中间件"，其核心思想是将各种异构工具统一抽象为可调用的服务，并提供统一的调用接口和响应处理机制。如图 15 所示，Scarffolds 框架由五个核心层次组成：基础架构层、解析层、管理层、工具层和配置层。各层之间通过明确定义的接口进行交互，形成了一个高内聚、低耦合的系统架构。

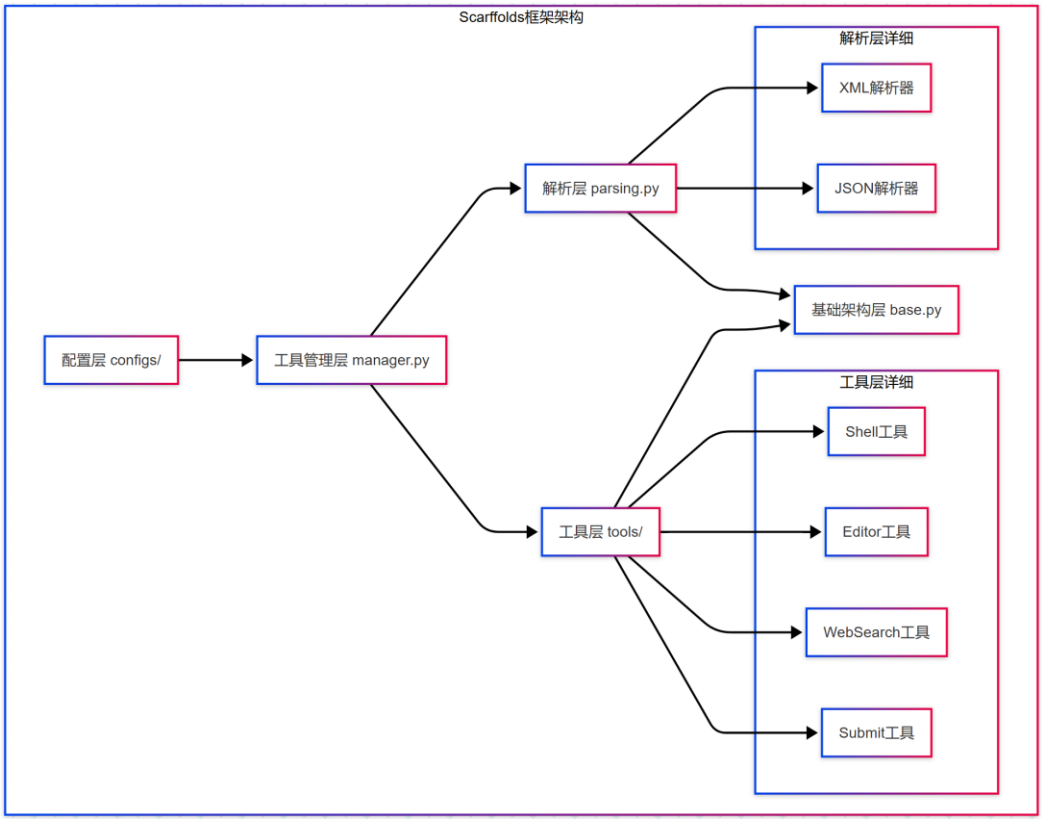


图 15 Scarffolds 框架整体架构图

进一步的，为了方便描述工具调用流程，我们将整个调用过程抽象为"请求-解析-执行-响应"的四阶段模型，如图 16 所示：

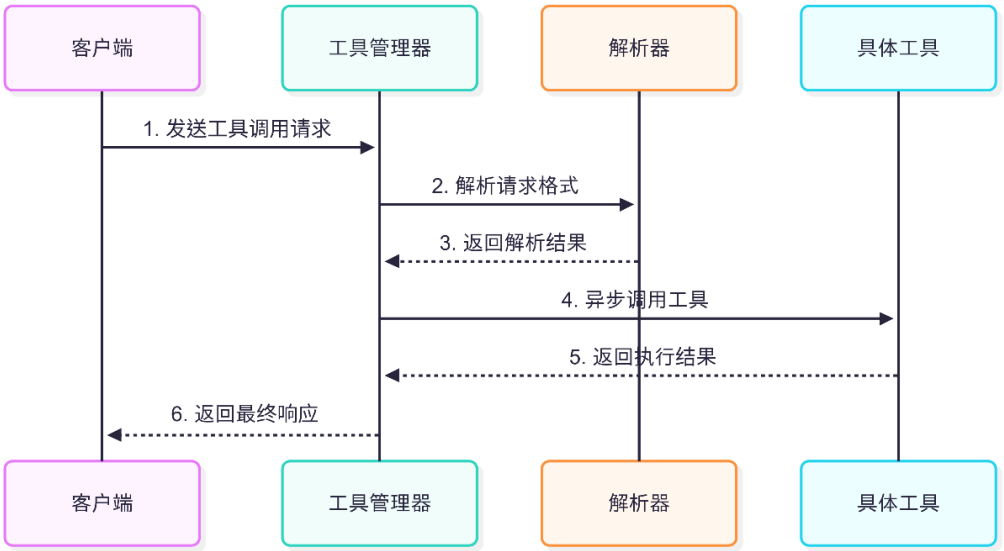


图 16 工具调用流程时序图

下面，我们开始介绍Scarffolds框架各模块的具体实现。

2. 解析层实现 (parsing.py)

解析层是 Scarffolds 框架的核心组件之一，负责将模型响应解析为可执行的工具调用指令。该层采用了策略模式设计，支持多种响应格式的解析，并提供了完善的参数验证机制。

3.1 抽象解析器设计

设计理念

AbstractParseFunction 采用了策略模式，定义了统一的解析接口。所有具体解析器都必须实现__call__方法，该设计具有以下特点：

- **策略可替换**：可以根据需要选择不同的解析策略
- **接口统一**：所有解析器遵循相同的调用约定
- **错误模板化**：提供标准化的错误信息模板

3.2 参数验证核心机制

在介绍具体解析器之前，我们先分析参数验证的核心函数 action_parsing，该函数是所有解析器的共同基础。

参数说明

变量名	数据类型	变量描述
ACTION_INSTANCE	DICT	待验证的动作实例
STRICT	BOOL	是否启用严格模式
TOOLS	LIST TOOLINSTANCE	可用工具列表

返回值说明

返回验证后的 ToolInstance 对象；验证失败时抛出 FormatError 异常。

验证逻辑说明

1. **工具存在性检查**：验证请求的工具是否在可用工具列表中
2. **严格模式处理**：在严格模式下，未知工具会导致验证失败
3. **参数合法性检查**：验证所有输入参数是否在工具的允许参数列表中
4. **对象构建**：验证通过后构建标准的 ToolInstance 对象

3.3 XML 格式解析器

支持的 XML 格式

```
<think>
思考过程内容
</think>
<tool_call>
{"name": "工具名称", "arguments": {"参数名": "参数值"}}
</tool_call>
```

关键实现细节

1. **正则表达式模式**：使用 `XML_RESPONSE_PATTERN` 精确匹配 XML 结构
2. **嵌套 JSON 处理**：tool_call 标签内容为 JSON 格式，需要二次解析
3. **错误信息层次化**：提供详细的格式错误提示信息

3.4 JSON 格式解析器

关键实现细节

1. **多层结构验证**：逐层验证 JSON 结构的完整性
2. **字段存在性检查**：确保所有必需字段都存在
3. **类型安全验证**：验证各字段的数据类型正确性

3. 工具管理层实现 (manager.py)

工具管理层是 Scarffolds 框架的核心控制中心，负责工具的注册管理、响应处理和异步调用协调。该层采用了管理者模式和工厂模式，为整个框架提供了统一的工具调用入口。

4.1 工具管理器核心设计

类定义

```python
class ToolManager:
    def __init__(self, config: ToolManagerConfig):
        self.tools = []
        if config.parsing_function == "xml":
            self.parsing_function = XMLThoughtActionParser(format=config.parsing_format)

        elif config.parsing_function == "json":
```

```
            self.parsing_function = JSONThoughtActionParser(format=config.parsing_format
)
        else:
            msg = f"parsing function : {config.parsing_function} is not implementated."

            raise ValueError(msg)
        self.strict = config.strict
```

核心属性说明

属性名	数据类型	属性描述
TOOLS	LIST TOOLINSTANCE	已注册的工具实例列表
PARSING_FUNCTION	ABSTRACTPARSEFUNCTION	响应解析器实例
STRICT	BOOL	严格模式标志

设计特点

1. 工厂模式应用：根据配置动态创建相应的解析器
2. 配置驱动：通过 ToolManagerConfig 统一管理配置参数
3. 解析器抽象：通过抽象接口支持多种解析策略

4.2 响应处理核心机制

response_postprocess 函数

函数原型

```
    def response_postprocess(self,  response) -> ActionParsingResponse
```

参数说明

变量名	数据类型	变量描述
RESPONSE	STR	模型原始响应字符串

返回值说明

返回 ActionParsingResponse 对象，包含解析结果或错误信息。

处理机制特点

1. 异常隔离：将解析层异常转换为标准响应格式

2. **状态标识**：通过 exit_status 明确标识处理结果
3. **信息保留**：完整保留错误信息用于调试和用户反馈

4.3 异步调用机制

function_calling 函数

函数原型

```
    async def function_calling(self,  action,  **kwargs) ->
ActionCallbackResponse
```

参数说明

变量名	数据类型	变量描述
ACTION	TOOLINSTANCE	待执行的工具实例
**KWARGS	DICT	额外的关键字参数

返回值说明

返回 ActionCallbackResponse 对象，包含执行结果或错误信息。

异步调用流程

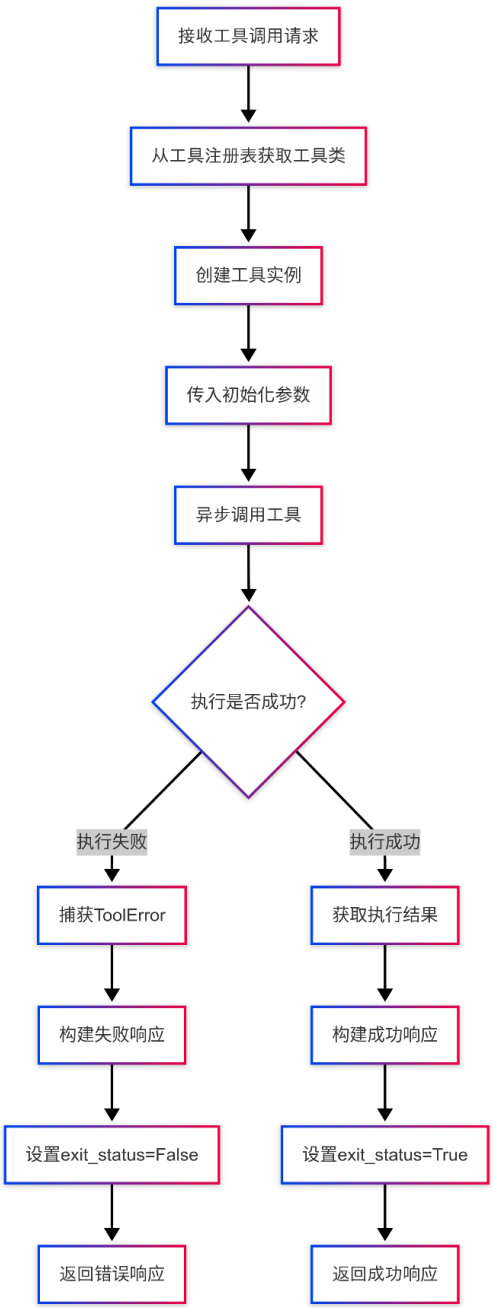


图 17 异步调用流程图

异步机制优势

1. **非阻塞执行**：工具调用不会阻塞主线程
2. **并发支持**：支持多个工具同时执行
3. **资源优化**：有效利用系统资源
4. **响应性提升**：提高整体系统响应速度

4.4 工具管理器完整调用流程

整个工具管理器的调用流程可以概括为以下时序图：

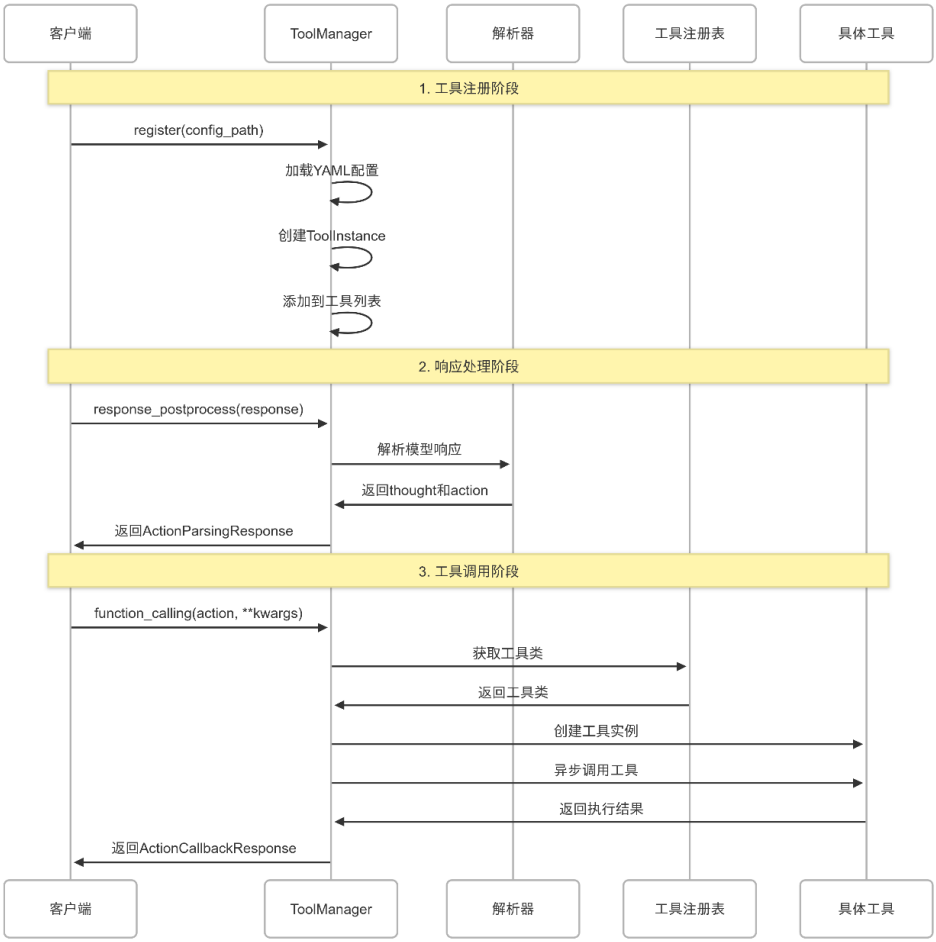


图 18 工具管理器完整调用时序图

调用流程说明

1. **注册阶段**：加载配置文件并注册工具到管理器
2. **解析阶段**：将模型响应解析为可执行的工具调用指令
3. **执行阶段**：异步调用具体工具并返回执行结果

这种分阶段的设计确保了系统的模块化和可维护性，同时提供了良好的错误处理和状态管理机制。

4. 工具层实现(tools)

5.1 工具层架构概述

Scarffolds 框架的工具层是整个系统的核心执行单元，负责与外部环境进行实际

交互。工具层采用模块化设计，每个工具都继承自统一的抽象基类，确保了接口的一致性和系统的可扩展性。

设计理念

工具层的设计遵循以下核心理念：

1. **统一接口**：所有工具继承自 `AbstractTool` 基类，提供一致的调用接口
2. **模块化设计**：每个工具作为独立模块，便于维护和扩展
3. **异步执行**：支持异步操作，提高系统并发性能
4. **错误隔离**：工具间错误互不影响，提高系统稳定性
5. **配置驱动**：通过 YAML 配置文件定义工具接口规范

工具注册机制

工具层通过 `__init__.py` 文件实现统一的工具注册机制，将所有可用工具映射到一个字典中，便于工具管理器进行动态调用。

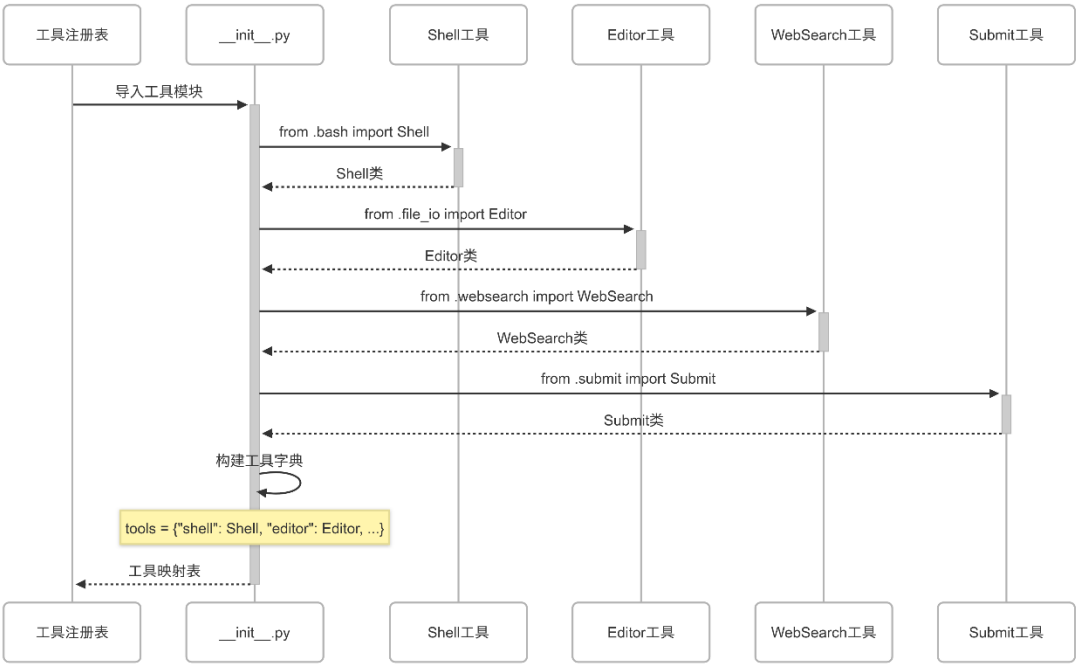


图 19 工具层初始化序列图

5.2 文件操作工具 Editor

设计理念

Editor 工具是 Scarffolds 框架中最核心的工具之一，其设计理念包括：

1. **统一文件操作接口**: 通过单一入口点支持多种文件操作命令
2. **环境适配性**: 同时支持本地文件系统和远程运行时环境
3. **精确行级操作**: 提供精确到行号的文件编辑能力
4. **错误边界检查**: 严格的参数验证和边界条件检查
5. **异步文件处理**: 采用异步 I/O 提高性能

5.3 命令执行工具 Shell

设计理念

Shell 工具是系统与外部环境交互的重要桥梁，其设计理念包括：

1. **命令执行抽象**: 提供统一的命令执行接口，屏蔽底层运行时差异
2. **交互式支持**: 支持交互式命令执行，如 Python 解释器等
3. **超时控制**: 防止命令执行时间过长影响系统性能
4. **期望输出匹配**: 支持期望输出模式匹配，用于交互式命令控制
5. **错误处理机制**: 完善的错误捕获和处理机制

参数说明

参数名称	数据类型	是否必需	默认值	描述
COMMAND	STRING	否	""	要执行的BASH命令
IS_INTERACTIVE	BOOLEAN	否	FALSE	是否以交互模式运行命令
EXPECT	STRING/LIST	否		期望的输出模式，用于交互式命令
TIMEOUT	INTEGER	否	5	命令执行超时时间（秒）

关键代码分析

1. 命令执行核心逻辑:

```
async def call_action(self, is_interactive, expect, command, timeout):
    command = self.convert_json_to_str(command)
    response = await self.runtime.run_in_session(BashAction(
        command=command,
        is_interactive_command=is_interactive,
        check="ignore",
        timeout=timeout,
        expect=expect
    ))
    return response
```

5.4 网络搜索工具 WebSearch

设计理念

WebSearch 工具为 AI 代理提供实时信息获取能力，其设计理念包括：

1. **多域名支持**：支持中国和美国两个搜索域名，提供全球化搜索能力
2. **异步 HTTP 请求**：使用 httpx 库进行异步网络请求，提高性能
3. **结果过滤优化**：智能过滤搜索结果，确保内容质量和相关性
4. **内容长度控制**：控制返回内容长度，避免信息过载
5. **错误处理机制**：完善的网络错误和 API 错误处理
6. **核心函数**

   WebSearch 工具包含以下核心函数：

- `__call__()`：主要调用入口，执行搜索请求
- `prepare_request()`：准备 HTTP 请求头和载荷
- `postprocess()`：处理搜索结果，过滤和格式化内容

参数说明

参数名称	数据类型	是否必需	默认值	描述
QUERY	**STRING**	**是**	**-**	**搜索查询字符串**
DOMAIN	**STRING**	**是**	**-**	**搜索域名：CN（中国）或US（美国）**

关键代码分析

1. 异步 **HTTP** 请求:

```
async with httpx.AsyncClient() as client:
    if domain == 'cn':
        response = await client.post(self.url_cn,  headers=headers,  data=payload)
    elif domain == 'us':
        response = await client.post(self.url_us,  headers=headers,  data=payload)
```

2. 结果后处理:

```
def postprocess(self,  response):
    webpages = response['data']['webPages']['value']
    raw_content = []
    for content in webpages:
        if len(raw_content) >= WEBSEARCH_WEB_NUM:
            break
        if len(content["summary"]) < WEBSEARCH_LENGTH_MINIMUM:
            continue
        raw_content.append(content)

    filtered_context = ""
    for idx,  content in enumerate(raw_content):
        url = content["url"]
        context = content["summary"][:WEBSEARCH_LENGTH_MAXIMUM]
        filtered_context += f"[{idx + 1}] {url}\n{context}\n\n"
    return filtered_context
```

5.5 提交工具 Submit

设计理念

Submit 工具是任务完成的标志性工具，其设计理念包括：

1. **版本控制集成**: 深度集成 Git 版本控制系统，自动生成代码变更补丁
2. **状态清理机制**: 自动清理测试补丁，确保提交的是最终代码状态
3. **原子性操作**: 通过组合 Git 命令确保提交操作的原子性
4. **变更追踪**: 完整追踪所有文件变更，生成详细的差异报告
5. **环境隔离**: 在独立会话中执行，避免与其他操作冲突

核心函数

Submit 工具包含以下核心函数：

- `__call__()`: 主要调用入口，执行完整的提交流程

关键代码分析

1. **Git 命令序列执行:**

```
response = await self.env.deployment.runtime.run_in_session(
    BashAction(
        command=" && ".join([
            f"cd {repo_path}",
            '''if [ -s "/root/test.patch" ]; then
            git apply -R < "/root/test.patch"
            fi''',
            "git add -A",
            "git diff --cached > /root/model.patch",
            "cat /root/model.patch"
        ]),
        session="submit"
    )
)
```

4.2.3 KFC-Agent 性能评估模块—— KERMIT

1. 模块整体架构

在本节中，我们将详细介绍评估模块 KERMIT 的技术实现。KERMIT 与工具调用模块 Scarffolds 深度耦合，用于多方面验证 KFC-agent 系统对于内核崩溃的修复能力。该模块通过构建自动化、并行化的 patch 验证流水线，确保每个 LLM agent 修改内核代码得到的补丁能在真实的 Linux kernel 仓库中被正确应用、编译、运行。KERMIT 融合了程序生成、环境重现、工具自动执行与语义推理评估等层次结构，实现对 agent debug 能力端到端的测评。各层次紧密协作，在真实环境中将 KFC-agent 修复内核崩溃的能力量化为编译通过率、执行成功率等多个指标，精确衡量系统的综合性能。评估模块四个关键层次如图 20：

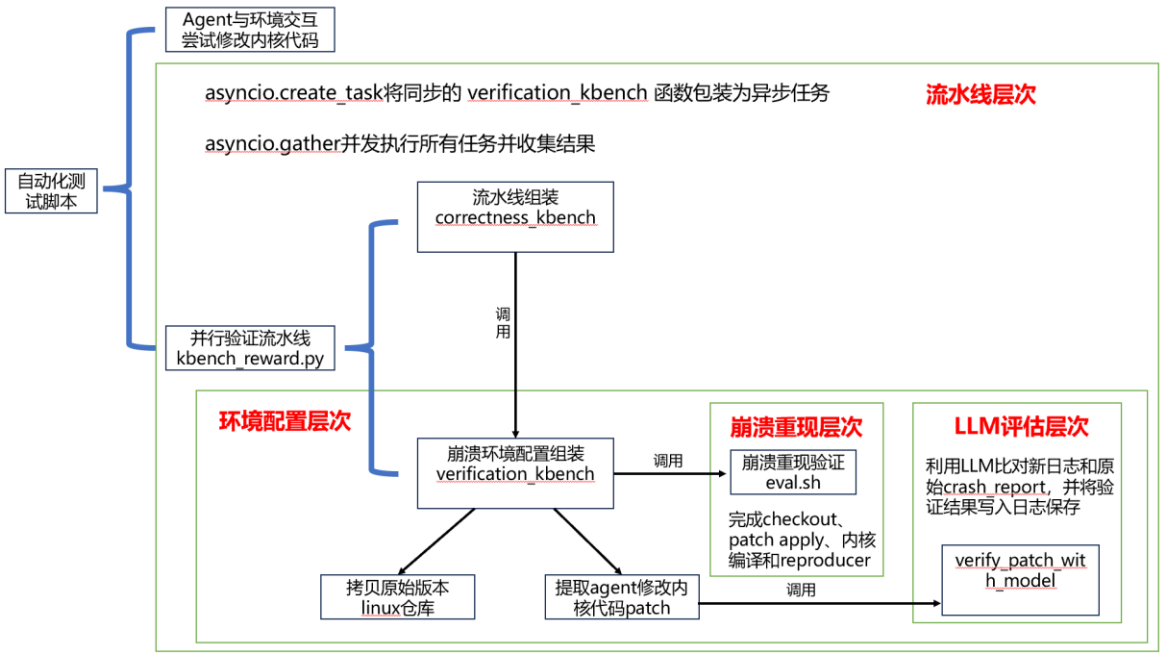


图 20 KERMIT 框架图

类似地，我们将四个层次的调用过程抽象为四阶段模型，如图 21：

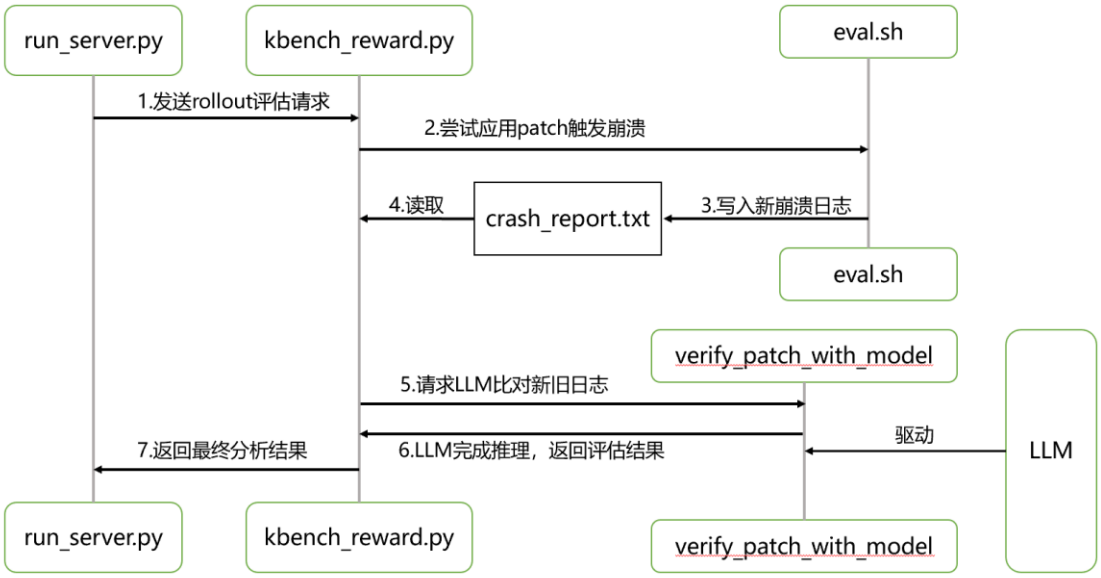


图 21 评估交互流程展示

下面将详细介绍KERMIT具体实现。该模块主要由kbench_rewards.py以及eval.sh两个脚本组成，其流程简图如图 22、图 23。

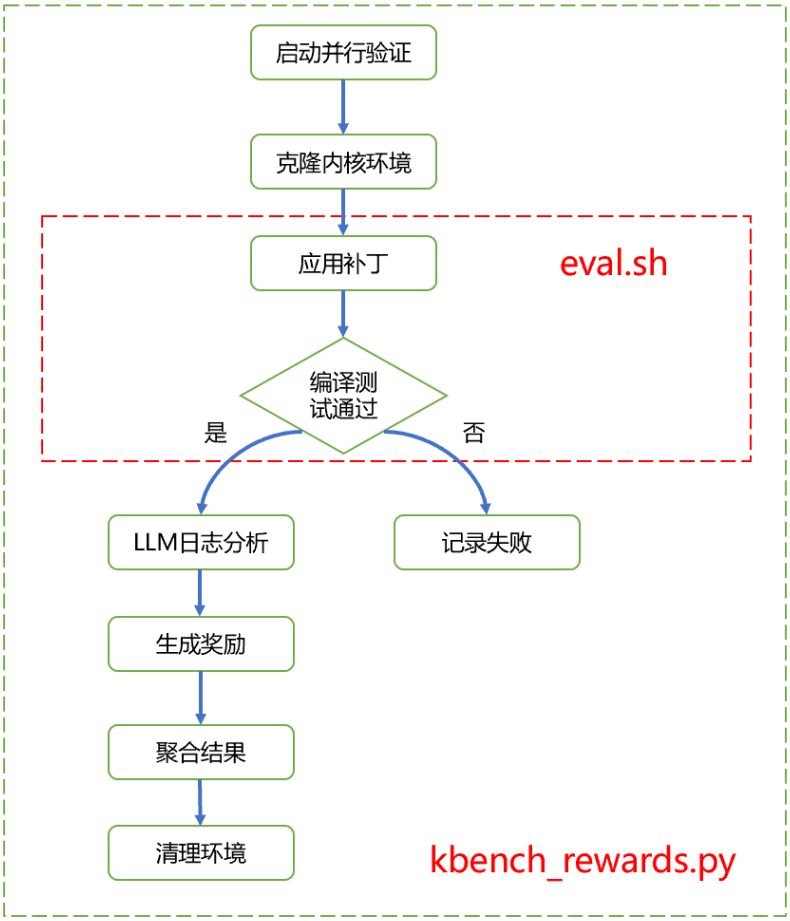


图 22 kbench_rewards.py 执行简图

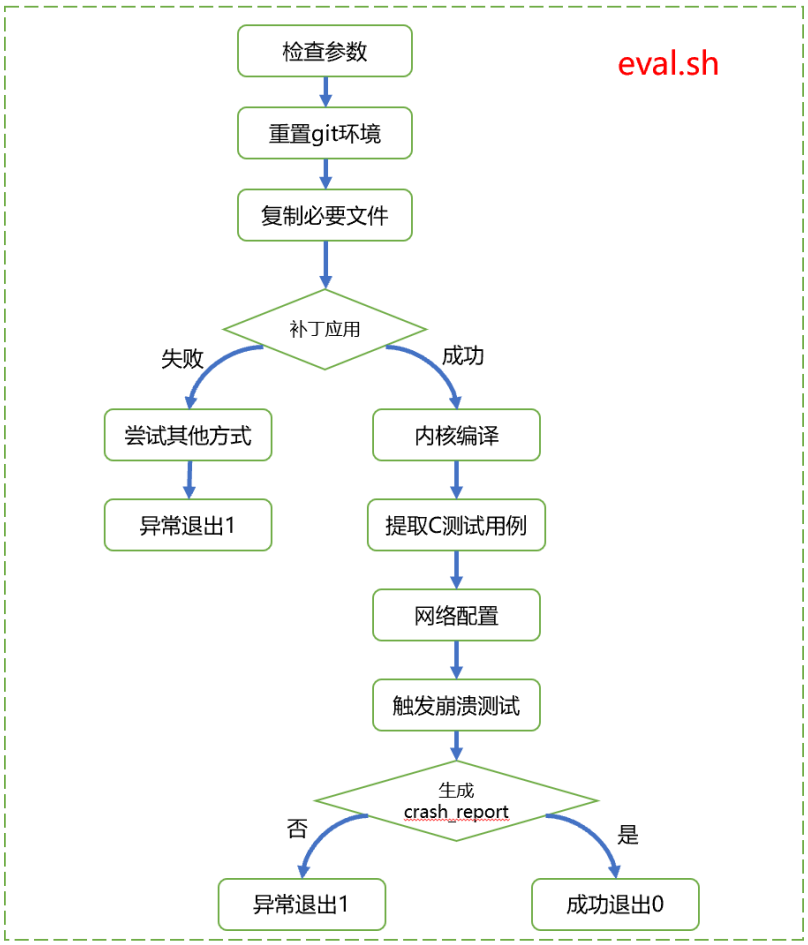


图 23 eval.sh 执行简图

2. 流水线和环境配置层实现（kbench_rewards.py)

流水线层次和环境配置层次均在kbench_rewards.py中实现，是KERMIT的最外层，负责整个评估流程的并行化处理以及验证环境准备。该层完成了环境配置和异步调度设计，实现高效精准评估。

2.1 环境配置流程设计

设计理念

采用"沙盒环境+原子化操作"的设计思想，通过临时目录隔离每个验证任务，该设计具有以下优势：

- 每次验证从干净内核代码开始，确保环境纯净性。
- 每次验证分配唯一的 ID，避免发生冲突。
- 保证清理环境，资源可回收。

核心函数

函数名：verification_kbench
参数： rollout: 包含环境元数据和工具响应的对象
        model_path: 模型路径
        batch_idx: 批次标识
        round_idx: 轮次标识
        log_config: {
            'log_path': 根日志目录,
            'project_name': 项目名称,
            'experiment_name': 实验名称
        }

返回值：int: 1-验证成功 0-验证失败
功能：完成环境准备及唯一性管理，构建 patch 文件，自动化执行 patch 脚本。

关键代码分析

1.环境初始化

```
# 生成隔离环境路径
unique_id = str(uuid.uuid4())
target_dir = os.path.join(kbench_dir,  f"linux-{base_commit}-{unique_id}")
# 保证基础目录存在
os.makedirs(kbench_dir,  exist_ok=True)
shutil.copytree("./Repositories/Raw/linux__linux",  target_dir)
```

2.补丁格式处理

```
# 编码标准化处理
if isinstance(patch_content,  bytes):
    patch_content = patch_content.decode('utf-8')
if not patch_content.endswith('\n'):
    patch_content += '\n'
```

3.资源清理释放

```
finally:
    if os.path.exists(target_dir):
        shutil.rmtree(target_dir)  # 递归删除临时目录
    if os.path.exists(patch_path):
        os.remove(patch_path)  # 删除补丁文件
```

执行流程

首先生成全局唯一的 UUID，并结合 base commit 创建独立验证目录 `kbench-verified/linux-<commit>-<uuid>`，并复制干净的 Linux 源码副本到该目录；接下来从 `rollout` 中获取 `agent` 提交的补丁文本，统一编码格式、补全换行并保存为 `.patch` 文件，完成评估环境初步搭建。

2.2 异步调度流程设计

设计理念

基于任务并行+结果聚合的模式，利用 Python asyncio 实现非阻塞任务创建、隐式协程调度和统一的错误处理。

核心函数

函数名：`sub_reward_correctness_kbench`
参数：`rollouts`: 待验证的 `rollout` 对象列表
`model_path`: 模型路径
`batch_indices`: 对应批次 ID 列表
`round_indices`: 对应轮次 ID 列表
`log_config`: 日志配置字典
返回值：`List[int]`: 每个 `rollout` 的验证结果列表
功能：为每个 `rollout` 调度异步验证，并发执行所有评估任务。

关键代码分析

1.任务并行化

```
tasks = []
for rollout, batch_idx, round_idx in zip(rollouts, batch_indices, round_indices):
    task = asyncio.create_task(verification_kbench(
        rollout=rollout,
        model_path=model_path,
        batch_idx=batch_idx,
        round_idx=round_idx,
        log_config=log_config
    ))
    tasks.append(task)
```

2.等待结果收集

```
rewards = await asyncio.gather(*tasks)
```

```
print(f"All instances run time: {time.time() - start:.2f} seconds")
```

执行流程

首先记录当前时间 start 以统计总耗时，然后遍历 rollouts、batch_indices 和 round_indices，通过 `asyncio.create_task` 为每个 `rollout` 调用 `verification_kbench` 并创建异步任务，将这些 `task` 聚集到 `tasks` 列表中；接着使用 `await asyncio.gather(*tasks)` 并发等待所有任务完成，并返回每个任务的执行结果；最后打印所有任务的总运行时间，用以评估并行效率，并将返回的奖励结果列表作为函数输出用于后续处理。

2.3 层次总流程图

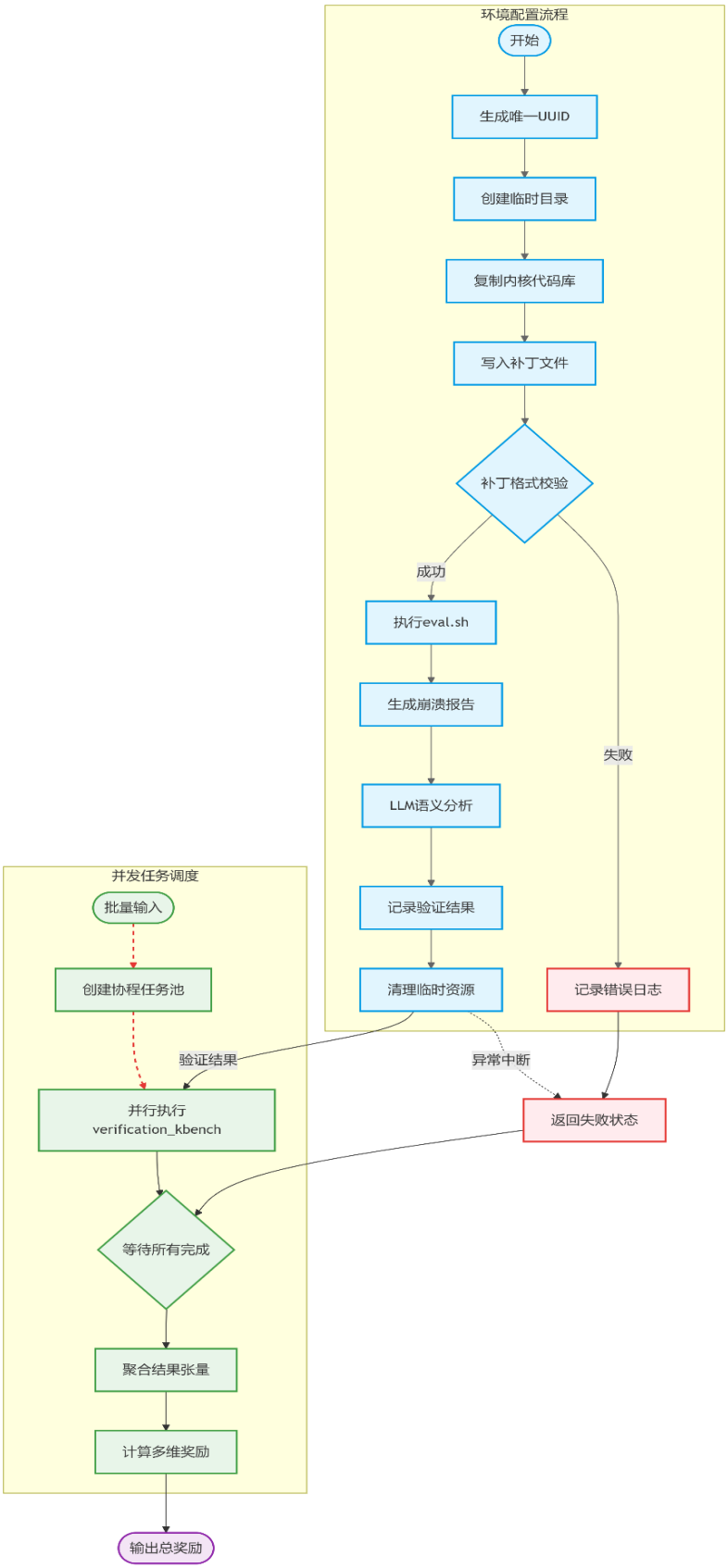


图 24 环境配置与异步流水线调度流程图

3. 崩溃重现层实现（eval.sh）

崩溃重现层次功能由 eval.sh 脚本负责实现，它实现了一套端到端自动化的内核补丁验证流程，通过编译、补丁应用和 LLM 评估系列操作，最终输出成功或失败的验证结果供后续评估使用。

3.1 进一步环境配置流程设计

设计理念

采用"沙盒环境+版本控制"模式，通过 Git 硬重置确保环境纯净的同时，锁定特定 commit，并采用强制文件校验保证文件完整性。

关键代码分析

```
# 三级环境清理
git reset --hard  # 消除所有本地修改
git clean -fdx    # 删除未跟踪文件（包括构建残留）
git checkout "$COMMIT_HASH"  # 切换到指定版本

# 文件校验
cp "$SOURCE_DIR/stretch-amd64.img" ./  # 基础镜像
cp "$SOURCE_DIR/local_network.sh" ./   # 网络配置脚本
[ ! -f "stretch-amd64.img" ] && exit 1 # 严格校验
```

执行流程

环境配置流程如图 25 所示：

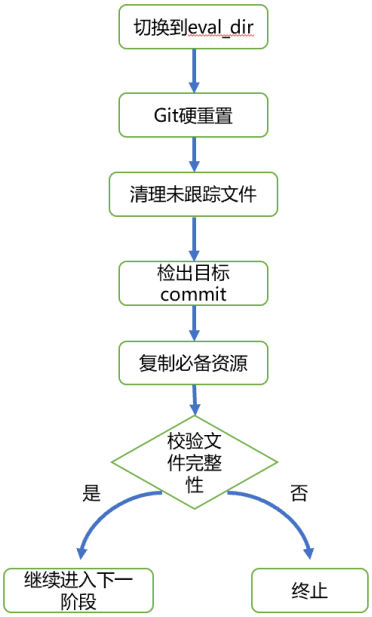


图 25 进一步配置评估环境流程图

3.2 应用补丁及内核配置编译流程设计

设计理念

采用"渐进式补丁应用+分层编译配置"，应用四级补丁应用回退策略，而内核配置包含“默认→扩展→远程”三级覆盖，并采用 cache 和多核 cpu 编译加速技术，优化测试过程。

关键代码分析

```
# 补丁应用
patch -p1 --forward < patch ||  # 标准应用
patch -p0 --forward < patch ||  # 降级尝试
git apply patch ||              # Git 原生方式
git apply -3 patch              # 三路合并

# 内核配置
make defconfig                  # 基础配置
make kvm_guest.config           # 虚拟机优化
wget "$CONFIG_LINK" -O config.new  # 下载定制配置
cat config.new >> .config       # 配置合并
make olddefconfig               # 解决冲突
```

执行流程

应用补丁及编译流程如图所示：

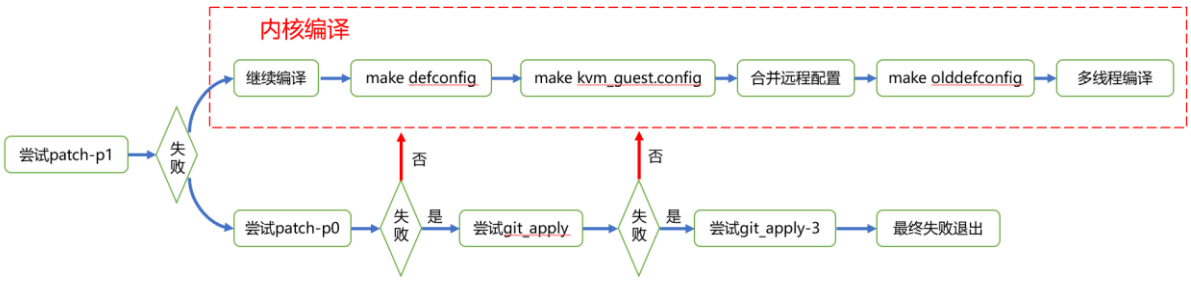


图 26 patch apply 与编译流程图

3.3 c_reproducer 构建日志流程设计

设计理念

我们构建了"动态获取+隔离测试"的崩溃重现系统：首先按需下载测试用例，并完成网络环境隔离后，实现自动化崩溃捕获逻辑。该层次可以有效避免存储膨胀和主机网络污染。

关键代码分析

```
# C 重现器生成
python3 extract_code.py "$C_LINK" -o "./C_repro.c"  # 从 URL 提取代码

# 网络隔离
echo "$SUDO_PASSWD" | ./local_network.sh  # 管道传递密码

# 崩溃测试
python3 get_crash.py --sudo-passwd "$SUDO_PASSWD"  # 安全执行
```

执行流程

崩溃重现流程如图 27 所示：

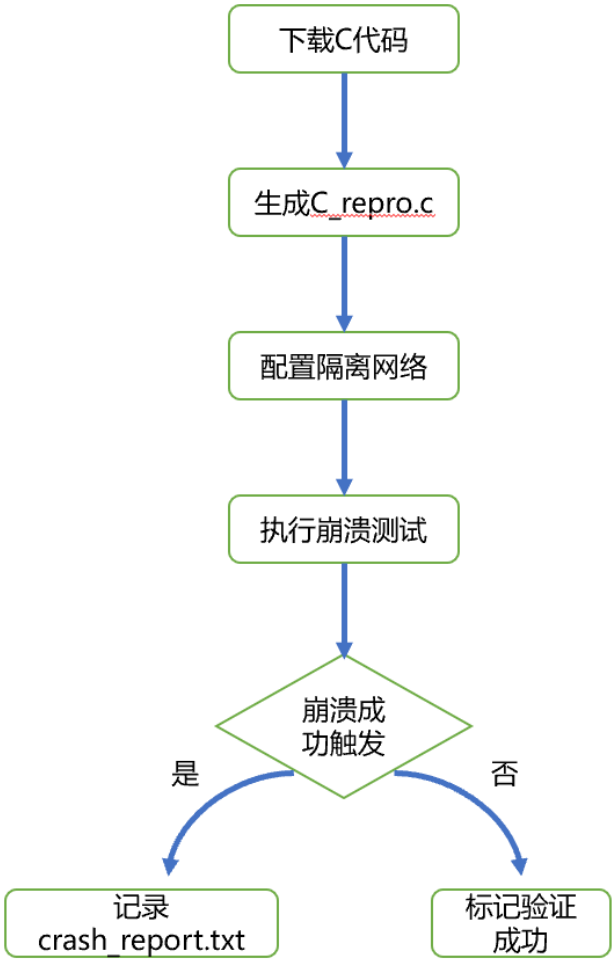


图 27 崩溃重现流程图

3.4 崩溃重现层次总流程图

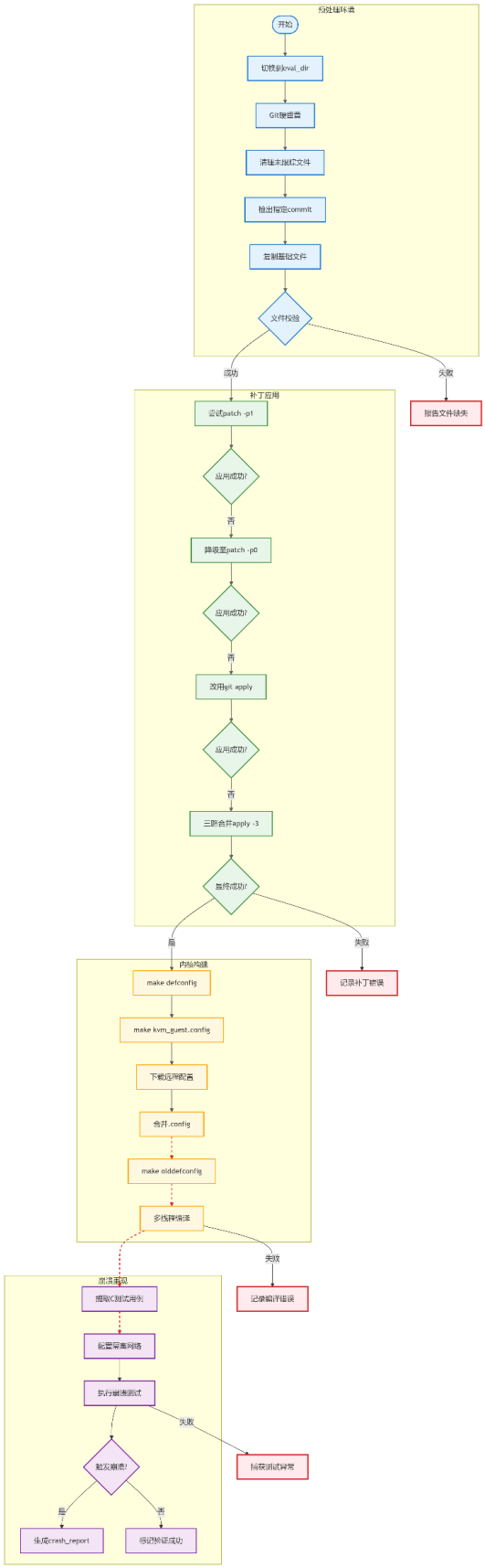


图 28 崩溃重现层次总流程图

4. LLM 评估层实现 (kbench_rewards.py)

设计理念

该层次通过设计结构化的LLM prompt，调用deepseek-chat模型，比对新旧crash_report对补丁正确性进行验证，并返回详细的分析过程。

关键代码分析

1.api 加载窗口

```
with open("configs/api_config.yaml",  "r") as f:
    api_config = yaml.safe_load(f)  # 安全加载 YAML 避免代码注入
api_key = api_config["api_key"]    # 密钥分离存储
base_url = api_config["base_url"]  # 支持多环境切换
```

2.结构化 Prompt 设计

```
prompt = f"""请判断以下问题是否被解决。你只能回答"yes"或"no"，并在<think></think>标签中给出你的推理过程。

原始问题：
{question}

新的 crash 报告：
{new_report}

请判断问题是否被解决："""
```

执行流程

LLM 评估的执行流程实际是 custom/service HTTP 协议实现的交互流程，如图 29 所示。

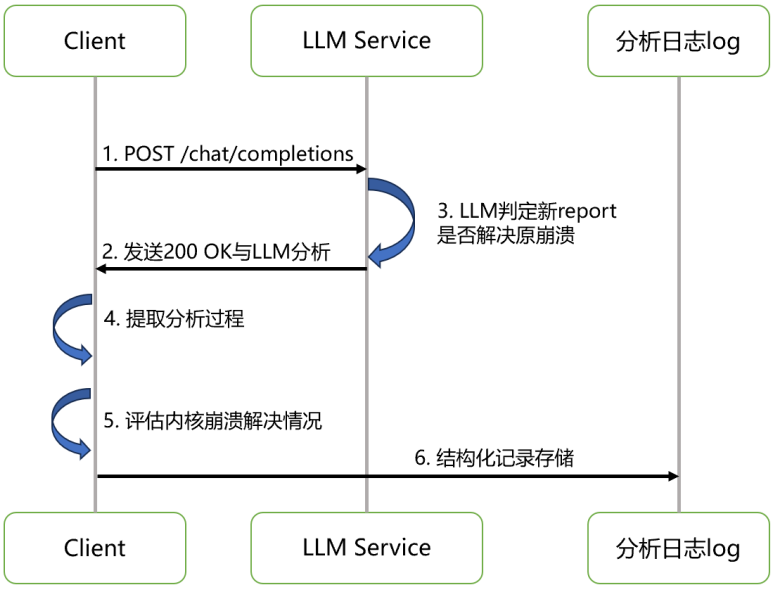


图 29 LLM 评估流程图

5 系统测试

5.1 测试准备

5.1.1 数据集来源与背景

本实验所使用的数据集来源于论文 *“KGym: A Platform and Dataset to Benchmark Large Language Models on Linux Kernel Crash Resolution”*[2]。该论文构建了一个名为 **kBench** 的专门数据集，目标是评估大型语言模型（LLMs）在真实世界中解决 Linux 内核崩溃问题的能力。

Linux 内核作为一个极其庞大和复杂的系统，其错误定位和修复任务具有高度挑战性。KGym 项目正是为了解决当前自动化补丁生成任务中缺乏现实性、结构化和系统性 benchmark 的问题而设计。

5.1.2 数据采集流程

KGym 团队的数据采集以 **Syzbot/Syzkaller** 工具生成的 Linux crash 报告为基础，通过以下流程构建了 kBench 数据集：

1. **崩溃收集：**
   - 研究人员从 Syzbot 上下载大量实际的 Linux crash 报告；
   - 这些报告包含堆栈跟踪、错误日志、相关 commit 链接和崩溃复现脚本等。
2. **复现验证：**
   - 使用 Crash Reproducer 工具链（包括 QEMU 虚拟机）对崩溃脚本进行复现验证；
   - 验证失败的 crash 会被过滤掉，仅保留那些可在特定环境下稳定复现的错误。
3. **数据清洗与构建：**
   - 对每个崩溃条目提取并组织为结构化格式，包括：
     - **错误代码（buggy code）；**
     - **官方修复补丁（ground-truth patch）；**
     - **崩溃日志（crash trace）；**
     - **重现脚本（repro script）；**
     - **上下文代码片段（上下文窗口自动提取）。**
4. **一致性校验与标注：**
   - 所有条目必须满足两个标准：
     1. **崩溃可复现；**
     2. **官方补丁能修复崩溃并通过编译/测试。**
   - 数据集中每条记录都由开发者原始提交信息和代码 diff 作为 ground truth。

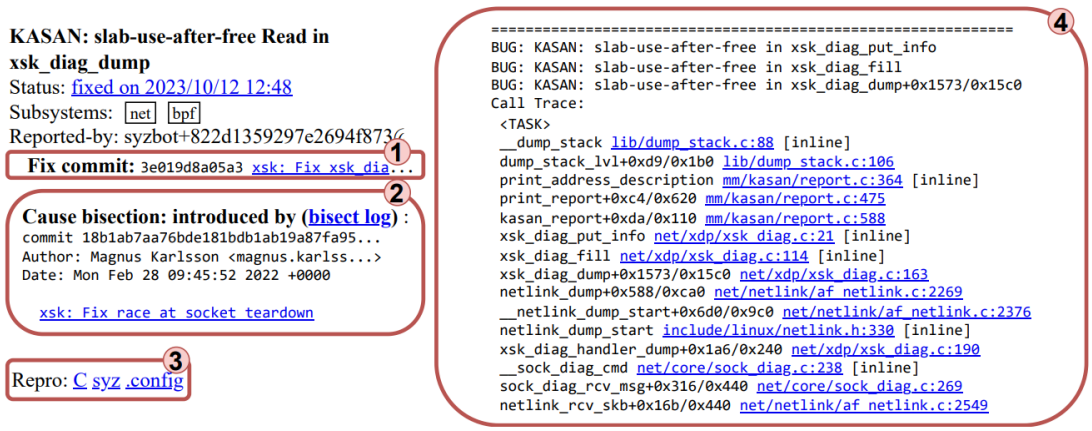


图 30 来自 Syzkaller 的一个内核漏洞示例 [Bug]

5.1.3 数据集特性

kBench 的主要特点包括：

特征	描述
数据量	数百条真实崩溃事件（覆盖多个内核版本）
多模态输入	包含堆栈 TRACE、REPRO 脚本、上下文代码等
多任务设置	支持无上下文/有上下文/带执行反馈的多种测试设置
高质量标注	所有补丁为开发者实际提交，确保质量
编译与运行验证	所有测试都包含真实编译和运行测试（PASS/FAIL）信息

KGym 还设计了一整套用于测试 LLM 的接口与验证标准（如是否编译通过、是否行为等效等），确保模型输出结果可用于实际应用场景中。

数据分类：

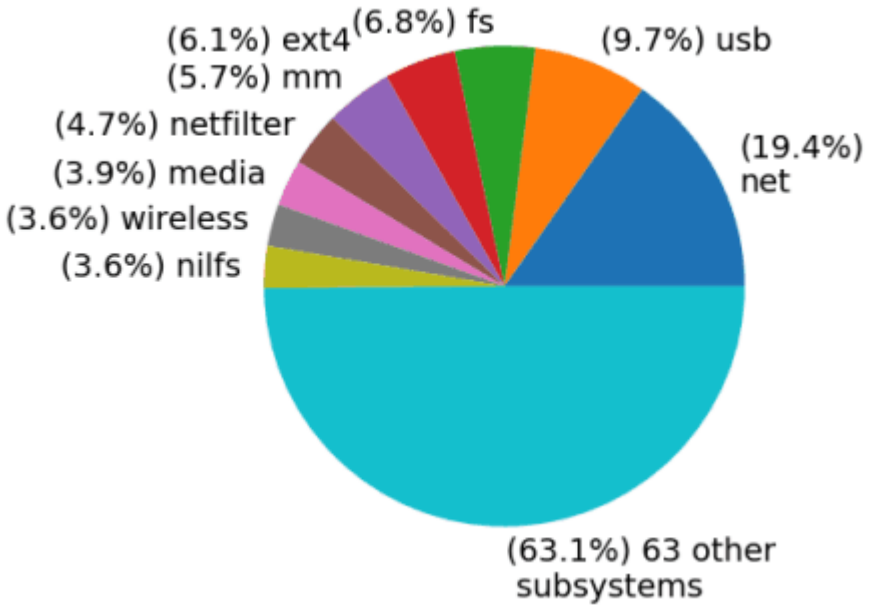


图 31 数据分类图

由图 31，可以看到该数据集在各个类别的 kernel crash 都有一定的占比，可见数据集覆盖范围广，数据多样性丰富。

数据集难度

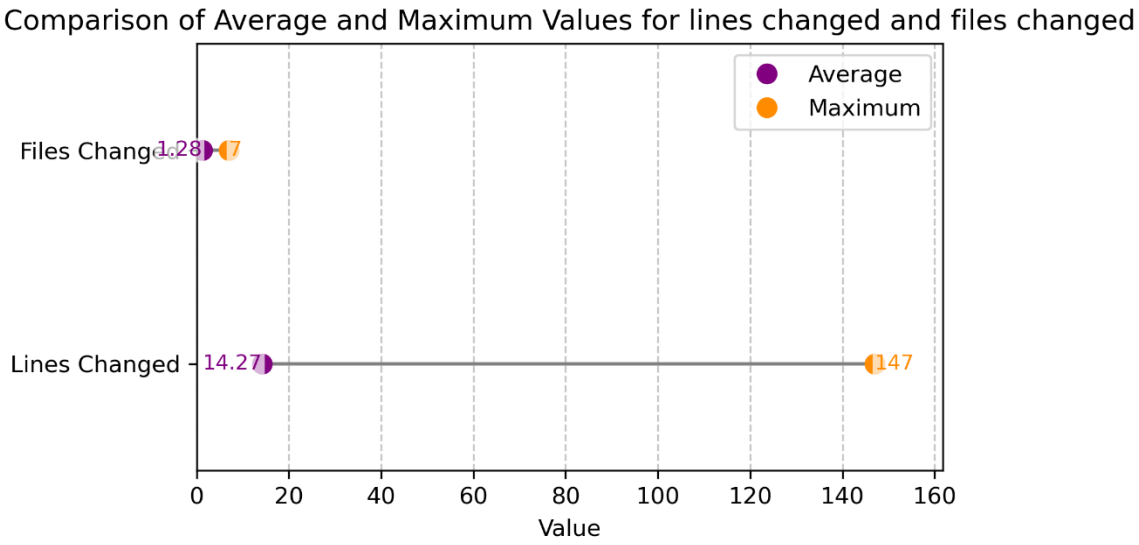


图 32 数据集难度

由图，根据统计信息，在 kbench 的 279 条数据中，数据的平均更改文件为 1.28 个，说明不少 patch 的修改仍然是跨文件的，跨文件数最多达到了 7 个，说明数据集对 LLM 能力仍有较大考验。修改的行数平均为 14.27 行，最多为 147 行，对 LLM 的代码理解能力亦有要求。

数据集信息量

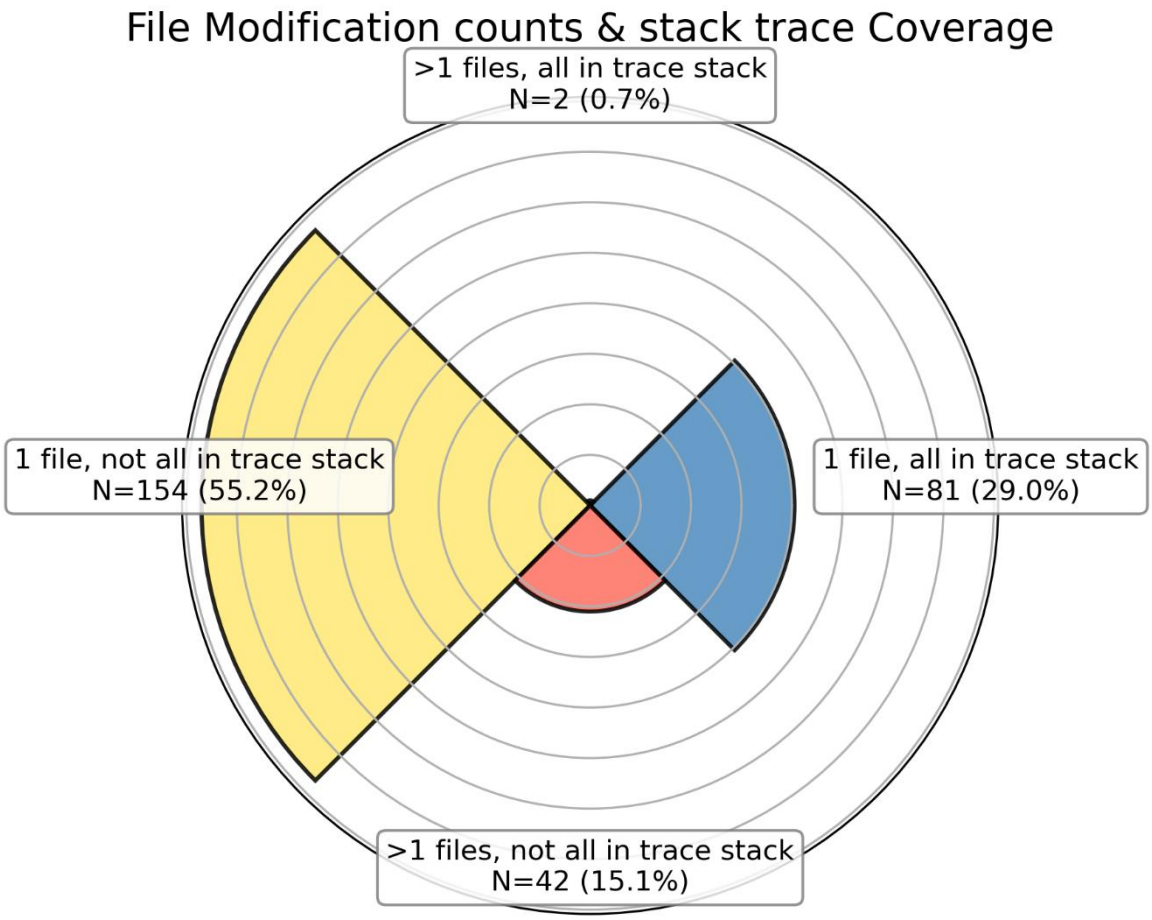


图 33 数据集信息量

由图，在对数据集进行分析的过程中，我们惊讶地发现，不同于 swe-agent 的 patch 修改往往就是 issue 中提到的位置，kernel crash 这一问题 patch 修改的文件往往不在 trace stack 即 crash report 中的调用栈内。上个图中修改文件内容不全在调用栈中的数据达到了 196 条，约 70%的数据量，可见对于代码调用关系的理解，构建理解 kernel 代码的 rag 框架对于正确生成 patch 具有重要的意义。而加了定位对执行效果有显著提升也从侧面印证了这一点。

5.1.4 测试环境配置

物理机器配置

配置项目	详细信息
处理器	Intel Core i7-8700 @ 3.20GHz (6 核 12 线程，最大睿频 4.6GHz)

内存	16GB DDR4
存储	系统盘: 233GB NVMe SSD 数据盘: 1.8TB HDD
操作系统	Ubuntu 23.04 (lunar)
内核版本	Linux 6.8.6
虚拟化支持	VT-x + KVM 硬件加速
虚拟化平台	QEMU 7.2.0 + KVM
编译工具链	GCC with ccache acceleration

虚拟机测试环境配置

配置项目	**参数设置**
CPU	2 个虚拟 CPU 核心 (-smp 2)
内存	4GB RAM (-m 4G)
CPU 类型	host passthrough (-cpu host)
虚拟化加速	KVM 硬件加速 (-enable-kvm)
网络配置	桥接网络 (-net bridge,br=br0)
虚拟机 IP	192.168.100.2/24
网关	192.168.100.1
存储	stretch-amd64.img (2.0GB, 基于 syzkaller 官方镜像)
存储接口	virtio-scsi (-device virtio-scsi-pci)
控制台	串口控制台 (console=ttyS0)
内核	自编译内核 (-kernel arch/x86/boot/bzImage)

5.2 测试结果

注：我们以 Kgym 作为 baseline 对比指标。由于实验成本差距悬殊，无法与 crashfixer 做规模相当的实验，因此不进一步对比

5.2.1 测评指标

在实验调研相关工作 swe-agent 和 kgym 以及实验过程中我们发现代码修复相关工作都存在 patch 编译失败的问题，在底层语言更甚，我们经过实验以及调研结果猜想，生成 patch 数变多能够提高编译率并提高执行正确率，考虑到编译时间成本，我们进行了 patch 数 n=1 以及 n=2 对编译率以及执行效果的影响的探索;此外由于 linux 仓库代码量巨大，我们认为定位修改的文件之后会对最终问题的解决效果有提升，因此初赛阶段我们还做了对定位效果的探索。综合上述探索并考虑到进行一轮 benchmark 完整测试的时间成本较大，初赛阶段我们进行了六组实验并确定了两个主要执行指标：

5.2.1.1 六组实验(n 指生成 patch 总数)

- deepseek-chat+kgym n=1
- deepseek-chat+kgym n=2
- deepseek-chat+KFC-agent n=1
- deepseek-chat+KFC-agent n=2
- deepseek-chat+KFC-agent+correct_location n=1
- deepseek-chat+KFC-agent+correct_location n=2

5.2.1.2 两个指标

- 执行正确率（solve）
- 编译通过率（apply）

5.2.2 总结果图

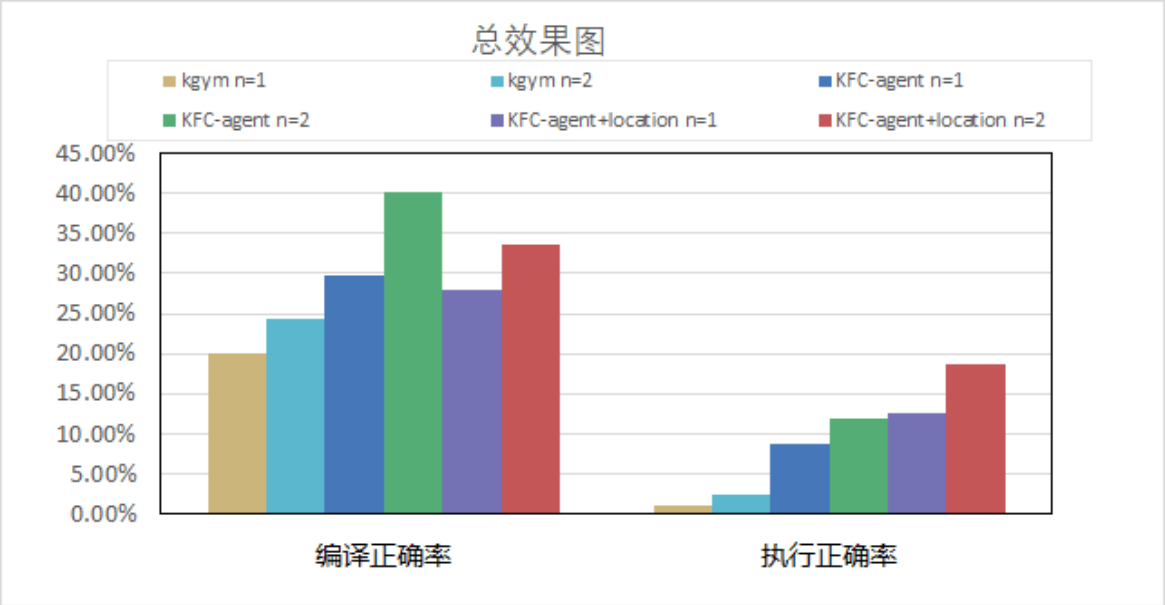


图 34 总结果图

实验组	编译成功率	执行成功率
kgym n=1	20.07%	1.08%
kgym n=2	24.37%	2.51%
KFC-agent n=1	29.75%	8.60%
KFC-agent n=2	40.14%	11.83%
KFC-agent+location n=1	27.96%	12.54%
KFC-agent+location n=2	33.69%	18.64%

如上图上表所示，我们的 agent 在效果上优于 kgym 这一 baseline，在生成 patch 数 n=2 时，执行正确率达到 11.83%，加上正确的定位信息后正确率达到 18.64%

5.2.3 patch 数对编译正确率与执行正确率的影响

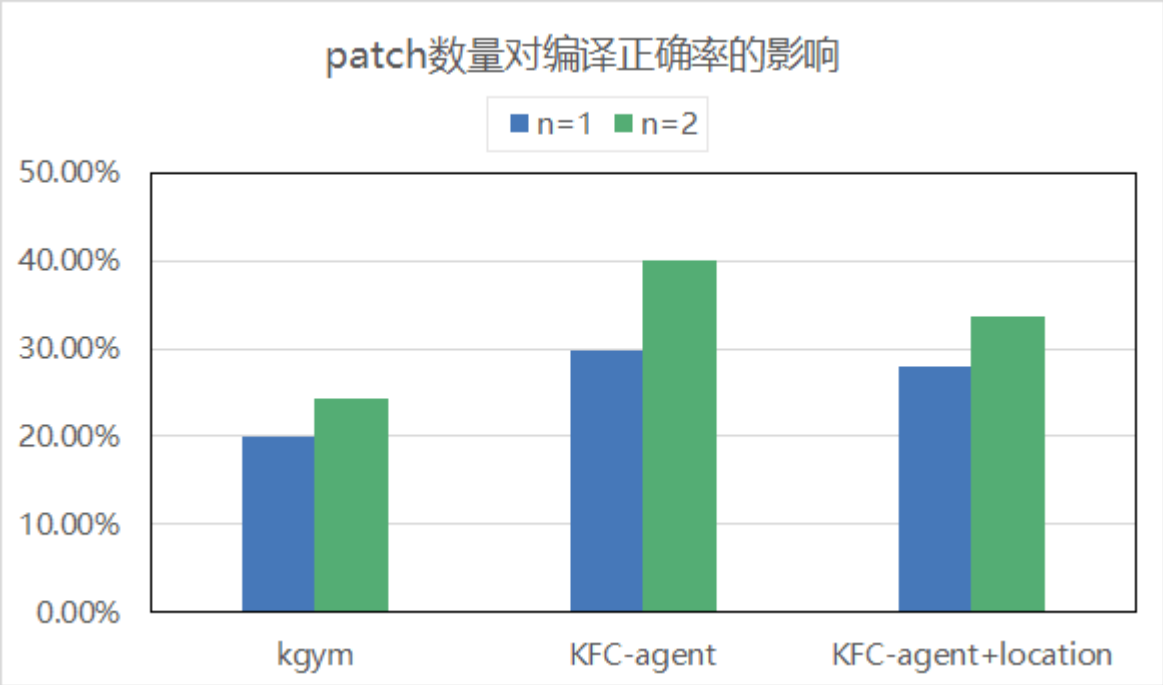


图 35 patch 数对编译正确率与执行正确率的影响

patch 数对编译正确率影响结果：

	N=1	N=2
kgym	20.07%	24.37%
KFC-agent	29.75%	40.14%
KFC-agent+location	27.96%	33.69%

如上图上表所示，在每轮实验中，增加 patch 数（即 n 增大），都能够提高当前问题中生成 patch 的编译成功率，能一定程度上缓解 agent 生成 patch 存在的编译易失败的问题

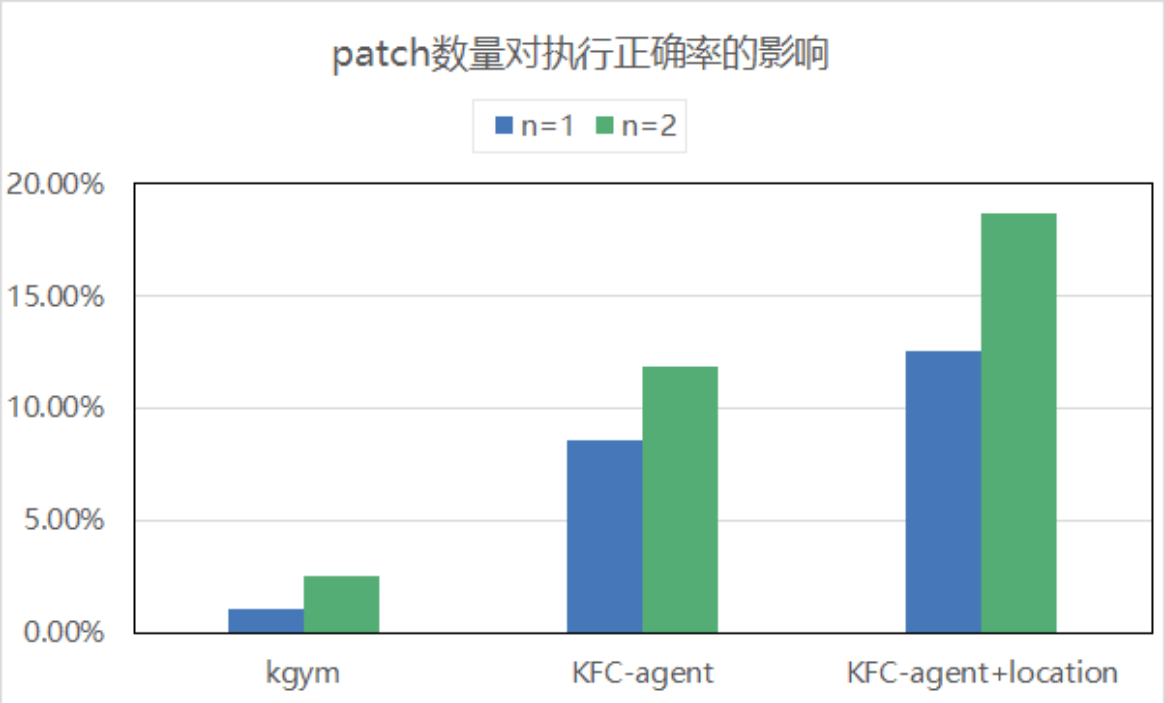


图 36patch 数量对执行正确率的影响

patch 数对执行正确率的影响结果：

	N=1	N=2
kgym	1.08%	2.51%
KFC-agent	8.60%	11.83%
KFC-agent+location	12.54%	18.64%

如上图上表所示，在每轮实验中，增大 patch 数（即 n 增大），都能够提高当前问题生成 patch 的解决问题成功率，这与相关工作 kgym 和 crashfixer 的实验情况一致，可以通过生成多个 patch，来对解决问题产生启发，增加问题的解决方案，提高解决问题成功率

5.2.4 定位对执行正确率的影响

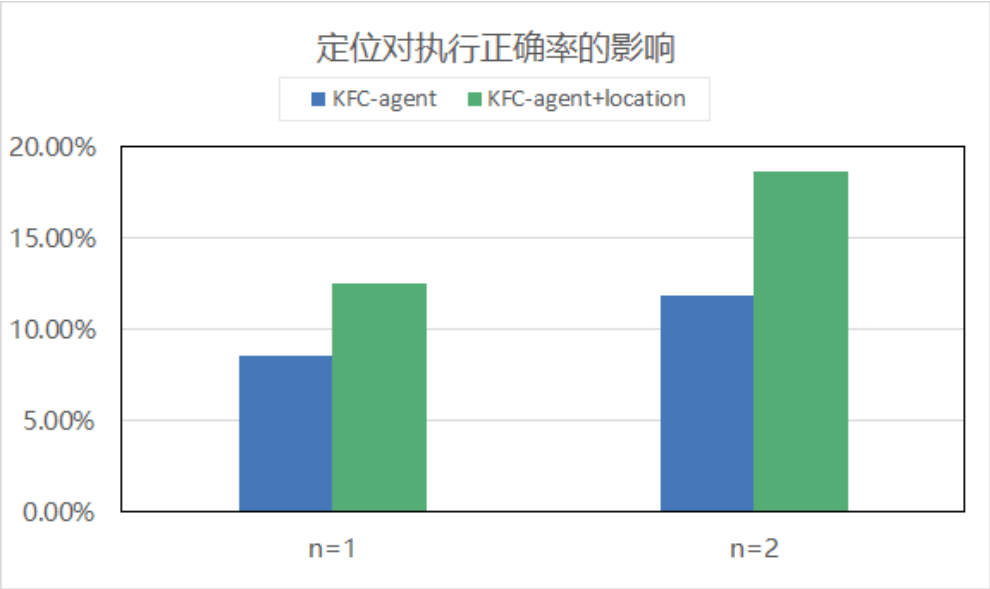


图 37 patch 数量对执行正确率的影响

定位对执行正确率的影响结果：

	KFC-AGENT	KFC-AGENT+LOCATION
n=1	8.60%	12.54%
n=2	11.83%	18.64%

如上图上表所示，在生成 patch 数量一致的情况下，加了准确定位之后 agent 生成的 patch 的执行正确率都会有提高，这说明有相关辅助定位的预处理或是工具有利于 agent 提高解决问题的成功率

6 初赛总结与决赛展望

6.1 初赛工作总结

在初赛阶段，我们围绕“如何利用大语言模型自动化地解决真实 Linux 内核崩溃问题”展开了系统性探索和工程实践。最终完成了以下关键工作：

- **搭建交互式环境（Environment）**

  基于 swe-agent 的 swe-rex 模块，构建了一个供 LLM 与挂载了内核源码的 docker 容器交互的环境，支持命令执行、文件读取修改等操作，有效支持多轮决策流程。

- **定义并实现工具接口（Tools）**

  结合实际开发者解决问题的行为抽象出四种工具：Shell、FileEditor、WebSearch、Submit。通过标准化输入输出格式，降低了模型调用门槛，提升了工具调用的可靠性。

- **设计交互式智能体（KFC-agent）**

  构建了一个具备上下文记忆和行动决策能力的多轮交互 agent，模拟人类从 crash trace 入手，逐步查询、修改、验证并提交补丁的全过程。

- **建立测评体系与执行验证**

  参考 kgym 和 syzkaller 的测评标准，搭建了本地自动评估系统，通过执行 trace 还原与补丁应用情况判断 patch 是否有效，成功实现自动化评估闭环，大大降低实验成本。

- **实验探索与对比分析**

  在 kbench 数据集上完成了六组实验，验证了 KFC-agent 在“执行成功率”“编译通过率”上相较 kgym 有显著提升。在 n=2 时，结合准确定位信息的执行成功率达 **18.64%**，远高于 kgym（2.51%）。

这些成果表明，多轮交互、环境支持和合理的工具接口设计能够显著缓解当前 LLM 在处理复杂系统软件错误问题时的上下文瓶颈和知识稀疏问题。

表6.1显示了对应赛题完成情况：

表6.1 赛题完成情况

实现内容	完成情况	说明
基础内容 1：借助 LLM 等手段自动化	全部完成	1.基于 swe-agent 中的 swe-rex 搭建一个环境，供智能体访问代码与探索 2. 构建了四个工具供模型使用，模拟开发者解决问题流程

常规 kdump 分析流程		3.设计一个与环境多轮交互的 agent，模仿开发者逐步收集信息解决问题 4.参考 kgym 与 syzkaller，完成了测评环境的搭建，跑通测试流程
基础内容 2：构建理解 kernel 代码的 rag 框架	初步完成（40%）	1. 对数据库所需数据进行了初步思考 2.调研测试了当前效果较好的代码 embedding 模型
进阶内容 1：结合历史问题数据（来源自 bugzilla、maillist 等……）进行问题分析	初步完成（30%）	1.查看并确立了对历史数据的选取标准 2.确立了历史数据对解决 kernel crash 问题的作用：用于理解问题思路，参与智能编排

6.2 创新点

KFC-Agent 在解决 Linux 内核崩溃问题的过程中，针对传统方法的痛点，从交互方式、工具集成、上下文处理到系统评估流程进行了系统性创新，主要体现在以下几个方面：

创新性多轮交互 Agent 工作流设计

区别于 kgym 等其他方法的一轮生成式修复方案，KFC-Agent 采用多轮交互工作流，模拟开发者“试探—反馈—修正”的真实修复过程。Agent 能够根据上下文信息采取新一轮行动，能够根据前一轮反馈动态调整修复策略，提升了系统应对复杂错误场景的鲁棒性和解释性。

抽象开发者行为为可调用工具，降低 LLM 使用门槛

项目提出将开发者在排查内核崩溃过程中的行为抽象为四种标准化工具（Shell、WebSearch、FileEditor、Submit），并设计了统一的接口格式，使 LLM 以低学习成本即可调用复杂系统命令，解决了特定内核工具调用门槛高的问题。

构建支持真实源码交互的环境，缓解上下文长度瓶颈

以 swe-rex 为基础改造环境模块，实现了代码仓库挂载、源码读取与修改、命令执行等功能，使模型可按需访问上下文内容，替代“整块代码直接输入”的 prompt 形式，缓解了 LLM 在处理超长源码时面临的长上下文问题。

构建标准化测评闭环，实现修复效果自动验证

项目设计了一套闭环的测评机制：自动收集 agent 输出的 patch → 应用于特定内核版本 → 使用 reproducer 镜像验证修复结果 → 判断成功/失败状态。该机制有效避免了仅凭 patch 语义判断正确性的片面性，实现了基于执行的 patch 正确性的客观评估。

探索 patch 生成数量与执行效果的关系

通过六组实验系统性分析了 patch 数量对编译率与执行成功率的影响，发现生成多个 patch 能显著提升 agent 成功修复率。该结论为后续 agent 动作决策、策略调度等模块提供了理论支撑和实践依据。

结合定位信息辅助生成，提高问题修复的精准度

创新性地引入人工标注的正确定位信息进行实验对比，发现结合定位可大幅提升修复成功率，验证了“先定位后修复”的流程设计合理性。该思路为未来引入自动定位模块打下基础，拓展了 LLM 在系统软件中的应用方式。

6.3 决赛展望

在初赛工作基础上，我们将进一步优化系统能力，具体计划如下：

- **增加定位模块，明确修改目标位置**
  当前版本中定位由人工标注辅助实现，后续将引入自动定位模块，使 agent 在决策过程中具备自动筛选相关文件和函数能力，提升 patch 精度。
- **构建 Rag 框架，增强代码语义理解能力**
  进一步完善代码理解方案，引入 Rag（Retrieval-augmented Generation）机制，结合代码 embedding 和相似代码检索，辅助 agent 理解 crash 背后深层结构逻辑。
- **引入历史知识，提升任务编排能力**
  整理并接入来自 Bugzilla、maillist 等平台的历史修复案例，使 agent 在执行前能先“阅读”类似问题解决思路，初步形成“先猜想、后探索”的智能任务分解路径。

我们期待在决赛阶段实现更强的智能 agent，并进一步提升系统的实用性、可扩展性与稳定性。

7 参考文献

[1] **Mathai A**, Huang C, Ma S, Kim J, Mitchell H, Nogikh A, Maniatis P, Ivančić F, Yang J, Ray B. *CrashFixer: A crash resolution agent for the Linux kernel*[J/OL]. 2025[2025-06-23]. https://arxiv.org/abs/2504.20412.

[2] **Mathai A**, Huang C, Maniatis P, Nogikh A, Ivančić F, Yang J, Ray B. *KGym: A Platform and Dataset to Benchmark Large Language Models on Linux Kernel Crash Resolution*[J/OL]. 2024[2025-06-23]. https://arxiv.org/abs/2407.02680.

[3] **Qiang R**, Zhuang Y, Li Y, Dingu S V K, Zhang R, Li C, Wong I S H, Yang S, Liang P, Zhang C, Dai B. *MLE-Dojo: Interactive Environments for Empowering LLM Agents in Machine Learning Engineering*[J/OL]. 2025[2025-06-23]. https://arxiv.org/abs/2505.07782.

[4] **Wang Z**, Wang K, Wang Q, Zhang P, Li L, Yang Z, Jin X, Yu K, Nguyen M N, Liu L, Gottlieb E, Lu Y, Cho K, Wu J, Fei-Fei L, Wang L, Choi Y, Li M. *RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning*[J/OL]. 2025[2025-06-23]. https://arxiv.org/abs/2504.20073.

[5] **Yang J**, Jimenez C E, Wettig A, Lieret K, Yao S, Narasimhan K, Press O. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*[J/OL]. 2024[2025-06-23]. https://arxiv.org/abs/2405.15793.

[6] **Zheng Y**, Fu D, Hu X, Cai X, Ye L, Lu P, Liu P. *DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-world Environments*[J/OL]. 2025[2025-06-23]. https://arxiv.org/abs/2504.03160.

[7] **Jin B**, Zeng H, Yue Z, Yoon J, Arık S Ö, Wang D, Zamani H, Han J. *Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning*[J/OL]. 2025[2025-06-23]. https://arxiv.org/abs/2503.09516.

[8] **Sheng G**, Zhang C, Ye Z, Wu X, Zhang W, Zhang R, Peng Y, Lin H, Wu C. *HybridFlow: A Flexible and Efficient RLHF Framework*[C/OL]. Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25). ACM, 2025: 1279 - 1297[2025-06-23]. http://dx.doi.org/10.1145/3689031.3696075.

[9] **SWE-agent Team.** *SWE-ReX: A Codebase for Agent-Environment Interaction in Software Engineering*[EB/OL]. [2025-06-23]. https://github.com/SWE-agent/SWE-ReX.

[10] **Chen M**, Li T, Sun H, Zhou Y, Zhu C, Wang H, Pan J Z, Zhang W, Chen H, Yang F, Zhou Z, Chen W. *ReSearch: Learning to Reason with Search for LLMs via Reinforcement Learning*[J/OL]. 2025[2025-06-23]. https://arxiv.org/abs/2503.19470.

[11] **Yang C**, Zhao Z, Zhang L. *KernelGPT: Enhanced Kernel Fuzzing via Large Language Models*[J/OL]. 2024[2025-06-23]. https://arxiv.org/abs/2401.00563.

[12] **Li X**, Yuan Z, Zhang Z, Sun Y, Zhang L. *Towards Large Language Model Guided Kernel Direct Fuzzing (SyzAgent)*[J/OL]. 2025[2025-06-23]. https://arxiv.org/abs/2503.02301.